UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 21: Parallel Sparse Methods

Volodymyr Kindratenko

4/9/2026

Course Reminders

- Project
  - Project milestone 2 is due this Friday
- Labs
  - Lab 8 (last lab) will be due next week
  - Today's lecture is needed for this lab
- Midterm 2
  - 5/5/26

Today's Objectives

- To learn the key techniques for compacting input data in parallel sparse methods for reduced consumption of memory bandwidth
  - Better utilization of on-chip memory
  - Fewer bytes transferred to on-chip memory
  - Better utilization of global memory
  - Challenge: retaining regularity

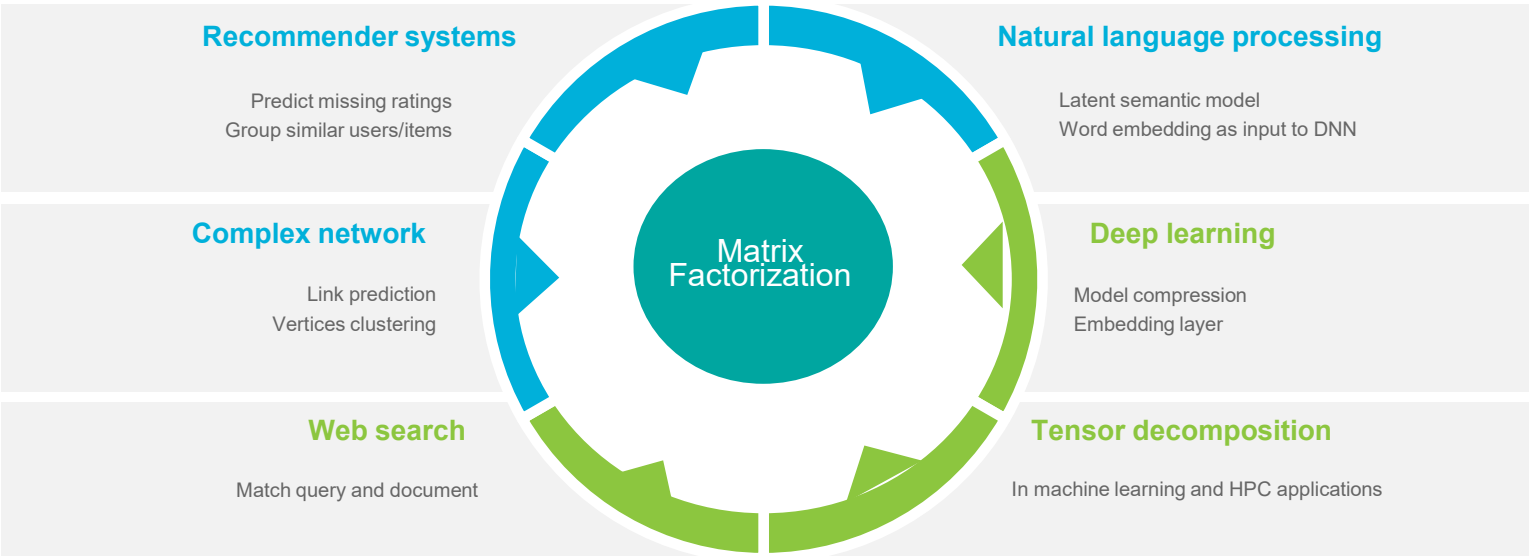
Sparse Matrix

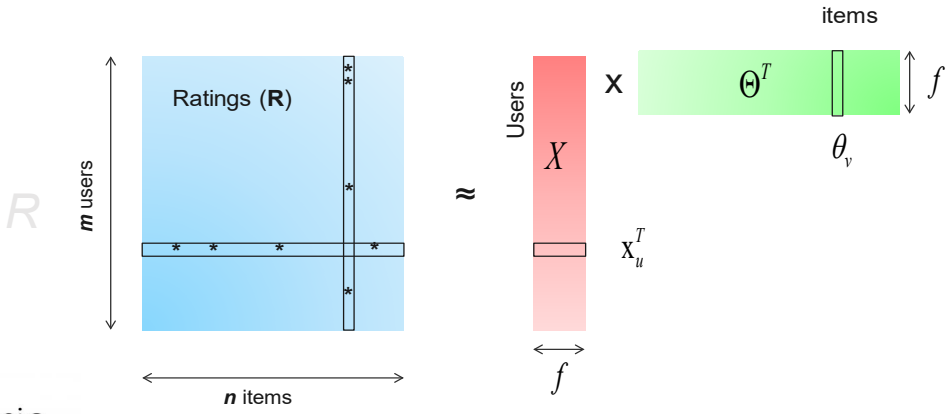
- Many real-world systems are sparse in nature
  - Linear systems described as sparse matrices
- Solving sparse linear systems
  - Iterative Conjugate Gradient solvers based on sparse matrix-vector multiplication is a common method
- Solution of PDE systems can be formulated into linear operations expressed as sparse matrix-vector multiplication

Sparse Matrix in Scientific Computing

Science Area	Number of Teams	Codes	Struct Grids	Unstruct Grids	Dense Matrix	Sparse Matrix	N-Body	Monte Carlo	FFT	PIC	Sig I/O
Climate and Weather	3	CESM, GCRM, CM1/WRF, HOMME	X	X		X		X			X
Plasmas/Magnetosphere	2	H3D(M),VPIC, OSIRIS, Magtail/UPIC	X				X		X		X
Stellar Atmospheres and Supernovae	5	PPM, MAESTRO, CASTRO, SEDONA, ChaNGa, MS-FLUKSS	X			X	X	X		X	X
Cosmology	2	Enzo, pGADGET	X			X	X				
Combustion/Turbulence	2	PSDNS, DISTUF	X						X		
General Relativity	2	Cactus, Harm3D, LazEV	X			X					
Molecular Dynamics	4	AMBER, Gromacs, NAMD, LAMMPS				X	X		X		
Quantum Chemistry	2	SIAL, GAMESS, NWChem			X	X	X	X			X
Material Science	3	NEMOS, OMEN, GW, QMCPACK			X	X	X	X			
Earthquakes/Seismology	2	AWP-ODC, HERCULES, PLSQR, SPECFEM3D	X	X			X				X
Quantum Chromo Dynamics	1	Chroma, MILC, USQCD	X		X	X					
Social Networks	1	EPISIMDEMICS									
Evolution	1	Eve									
Engineering/System of Systems	1	GRIPS,Revisit						X			
Computer Science	1			X	X	X			X		X

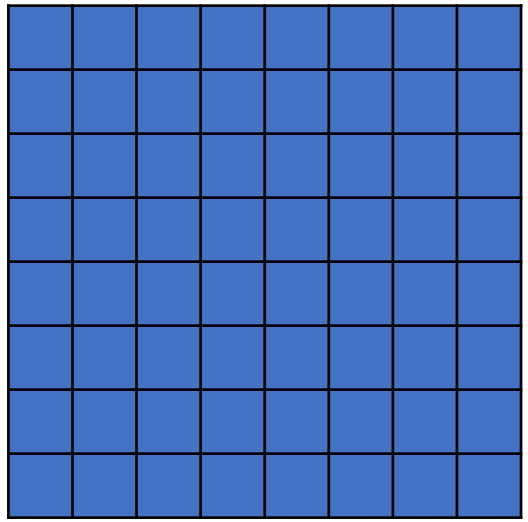
Sparse Matrix in Analytics and AI

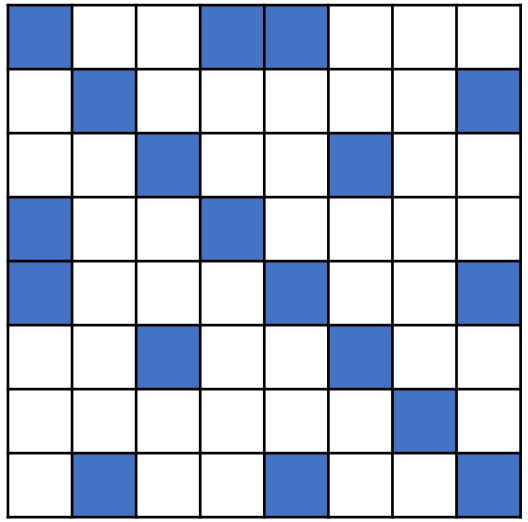




Dense vs. Sparse Matrices

- A **dense matrix** is one where the majority of elements are not zero
- A **sparse matrix** is one where many elements are zero





Opportunities:

Do not need to allocate space for zeros (save memory capacity)
Do not need to load zeros (save memory bandwidth)
Do not need to compute with zeros (save computation time)

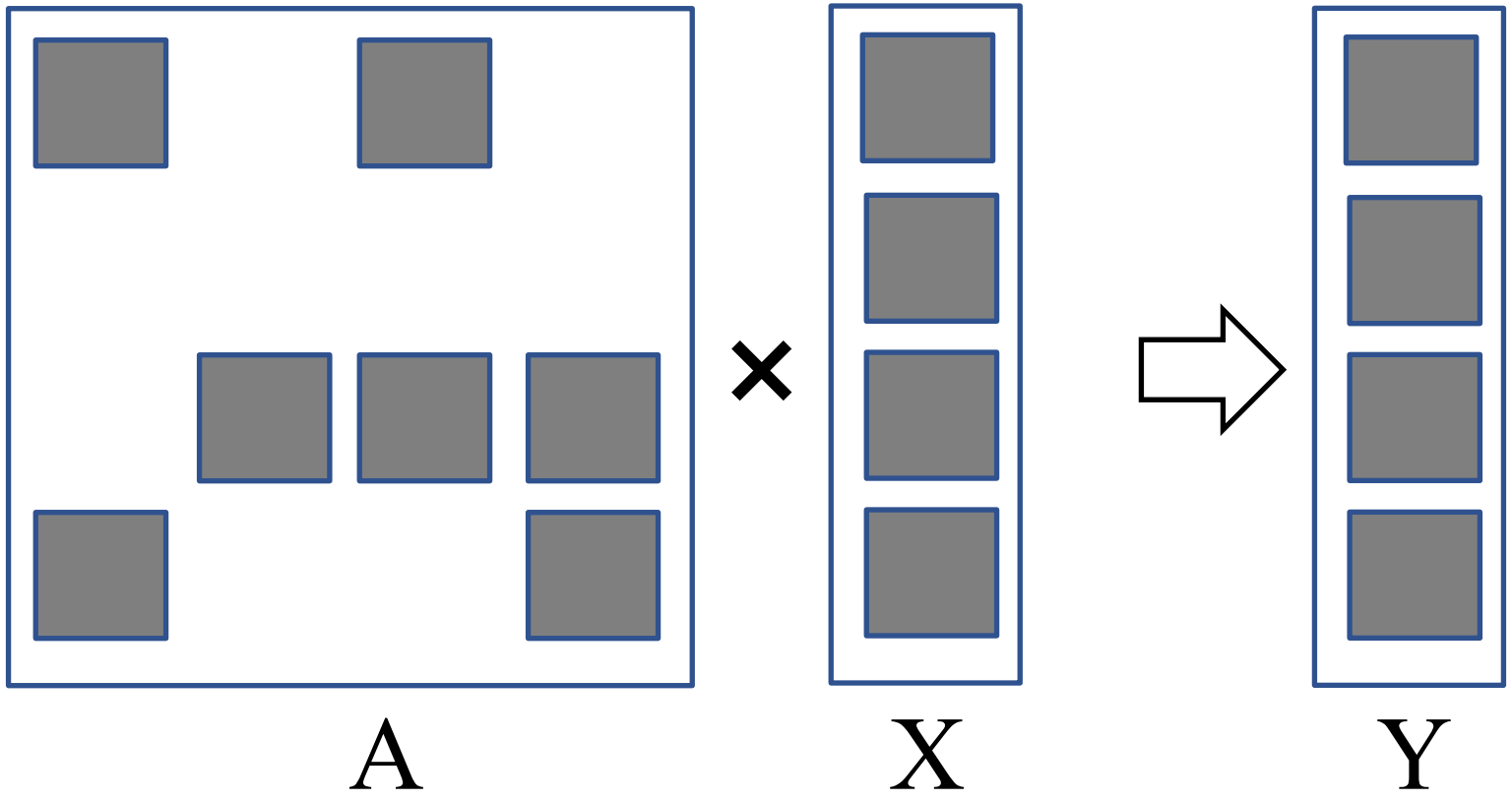
Sparse Matrix Storage Formats

- Many storage formats for sparse matrices:
  - Coordinate Format (COO)
  - Compressed Sparse Row (CSR)
  - ELLPACK Format (ELL)
  - Jagged Diagonal Storage (JDS)
  - …

- Format design considerations:
  - Space efficiency (memory consumed)
  - Modifiability (ease of adding/reordering elements)
  - Accessibility (ease of finding desired data)
  - Memory access pattern (enabling coalescing)
  - Load balance (minimizing control divergence)

Sparse Matrix-Vector Multiplication (SpMV)

- We will use Sparse Matrix-Vector multiplication (SpMV) as an example to study different formats



Challenges

- Compared to dense matrix multiplication, SpMV
  - Is irregular/unstructured
  - Has little input data reuse
  - Benefits little from compiler transformation tools

- Key to maximal performance
  - Maximize regularity (by reducing divergence and load imbalance)
  - Maximize DRAM burst utilization (layout arrangement)

Coordinate Format (COO)

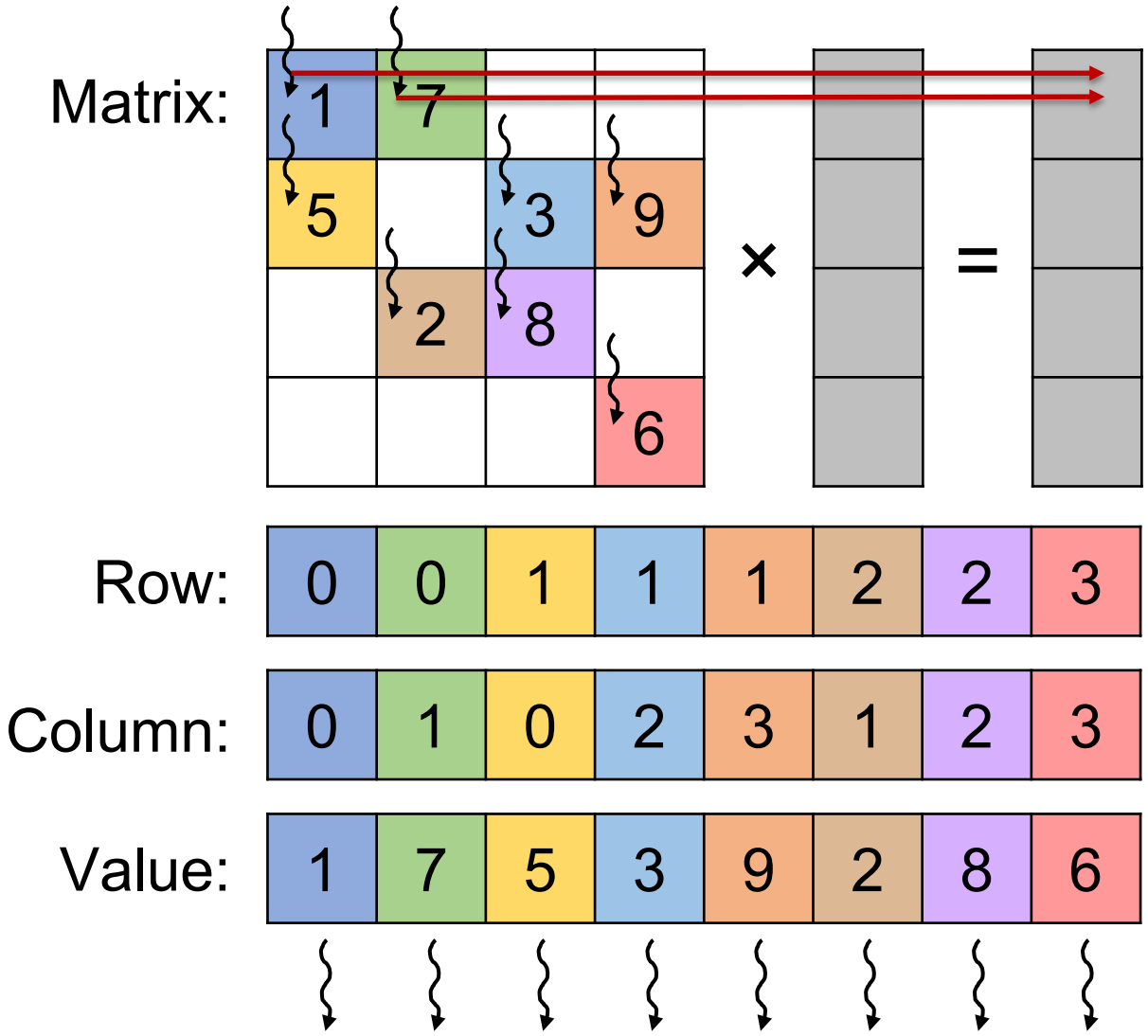
- Store every nonzero along with its row index and column index

Matrix:

1	7		
5		3	9
	2	8	
			6

Row:	0	0	1	1	1	2	2	3
Column:	0	1	0	2	3	1	2	3
Value:	1	7	5	3	9	2	8	6

SpMV/COO Parallelization



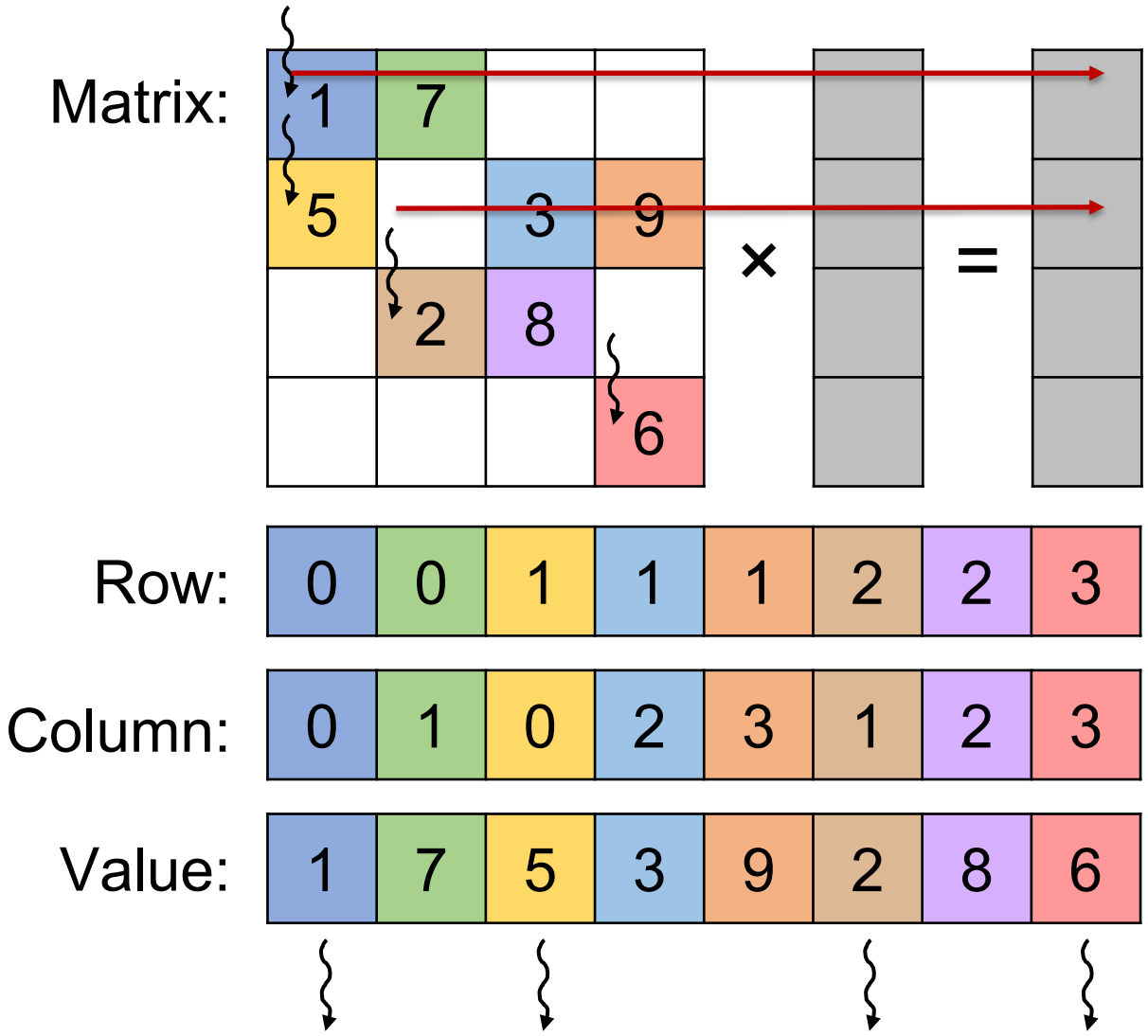
Multiple threads writing to the same output
(need atomic operations)

Parallelization approach:
Assign one thread per nonzero

SpMV/COO Code

```
__global__ void spmv_coo_kernel(COOMatrix cooMatrix,
                                float* inVector, float* outVector) {
    unsigned int i = blockIdx.x*blockDim.x + threadIdx.x;
    if(i < cooMatrix.numNonzeros) {
        unsigned int row = cooMatrix.rowIdxs[i];
        unsigned int col = cooMatrix.colIdxs[i];
        float value = cooMatrix.values[i];
        atomicAdd(&outVector[row], inVector[col]*value);
    }
}
```

SpMV/COO Parallelization – Another Strategy



Single thread writing to the same output

Parallelization approach:
Assign one thread per row

COO Tradeoffs

- Advantages:
  - Modifiability: easy to add new elements to the matrix, nonzeros can be stored in any order
  - Accessibility: given nonzero, easy to find row and column
  - SpMV/COO memory accesses to the input matrix are coalesced
  - SpMV/COO has no control divergence

- Disadvantage:
  - Accessibility: given a row or column, hard to find all nonzeros (need to search)
  - SpMV/COO memory accesses to the output vector require atomic operations

COO Allows Reordering of Elements

			Row 0	Row 2	Row 3	
Nonzero values	data[7]	{	3, 1,	2, 4, 1,	1, 1	}
Column indices	col_index[7]	{	0, 2,	1, 2, 3,	0, 3	}
Row indices	row_index[7]	{	0, 0,	2, 2, 2,	3, 3	}

Nonzero values	data[7]	{ 1 1, 2, 4, 3, 1 1 }
Column indices	col_index[7]	{ 0 2, 1, 2, 0, 3, 3 }
Row indices	row_index[7]	{ 3 0, 2, 2, 0, 2, 3 }

Compressed Sparse Row (CSR)

- Store nonzeros of the same row adjacently and an index to the first element of each row

Matrix:

1	7		
5		3	9
	2	8	
			6

RowPtrs:

0	2	5	7	8

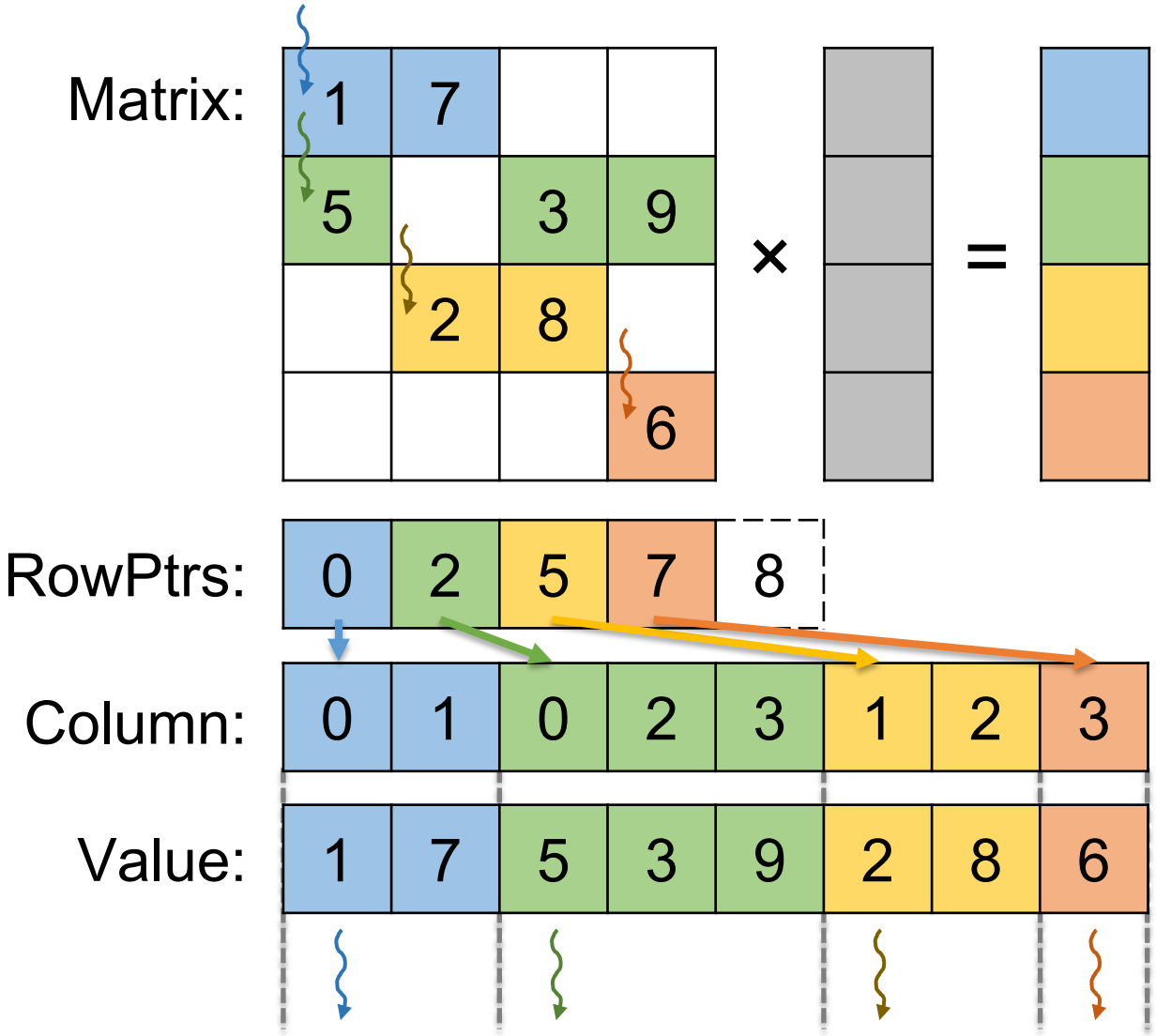
Column:

0	1	0	2	3	1	2	3

Value:

1	7	5	3	9	2	8	6

SpMV/CSR Parallelization



Parallelization approach:
Assign one thread to loop over each input row sequentially and update corresponding output element

SpMV/CSR Code

```
__global__ void SpMV_CSR(int num_rows, float *data,
            int *col_index, int *row_ptr, float *x, float *y)
 {
     int row = blockIdx.x * blockDim.x + threadIdx.x;
     if (row < num_rows) {
       float dot = 0;
       int row_start = row_ptr[row];
       int row_end =   row_ptr[row+1];
       for (int elem = row_start; elem < row_end; elem++)
           dot += data[elem] * x[col_index[elem]];
       y[row] = dot;
     }
 }
```

CSR Tradeoffs (versus COO)

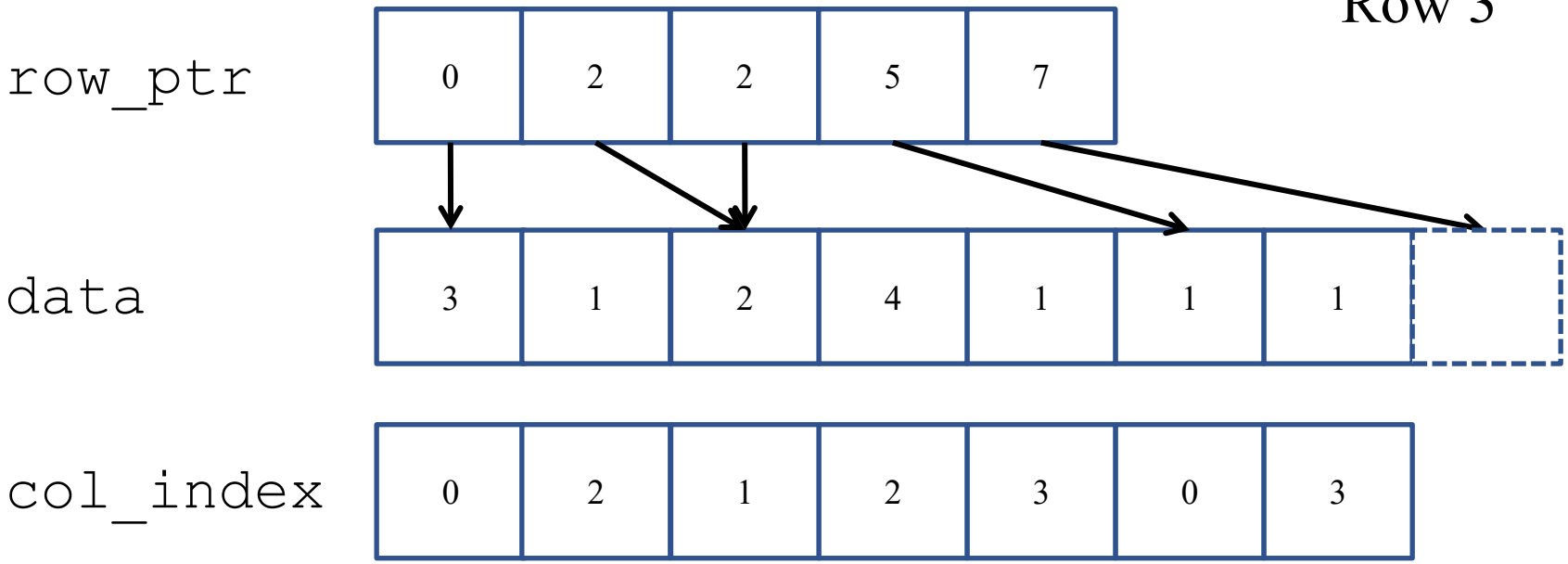
- Advantages:
  - Space efficiency: row pointers smaller than row indexes
  - Accessibility: given a row, easy to find all nonzeros
  - SpMV/CSR memory accesses to the output vector are coalesced and do not require atomics

- Disadvantage:
  - Modifiability: hard to add new elements to the matrix
  - Accessibility: given nonzero, hard to find row; given a column, hard to find all nonzeros
  - SpMV/CSR memory accesses to the input matrix are not coalesced
  - SpMV/CSR has control divergence

CSR Kernel Control Divergence

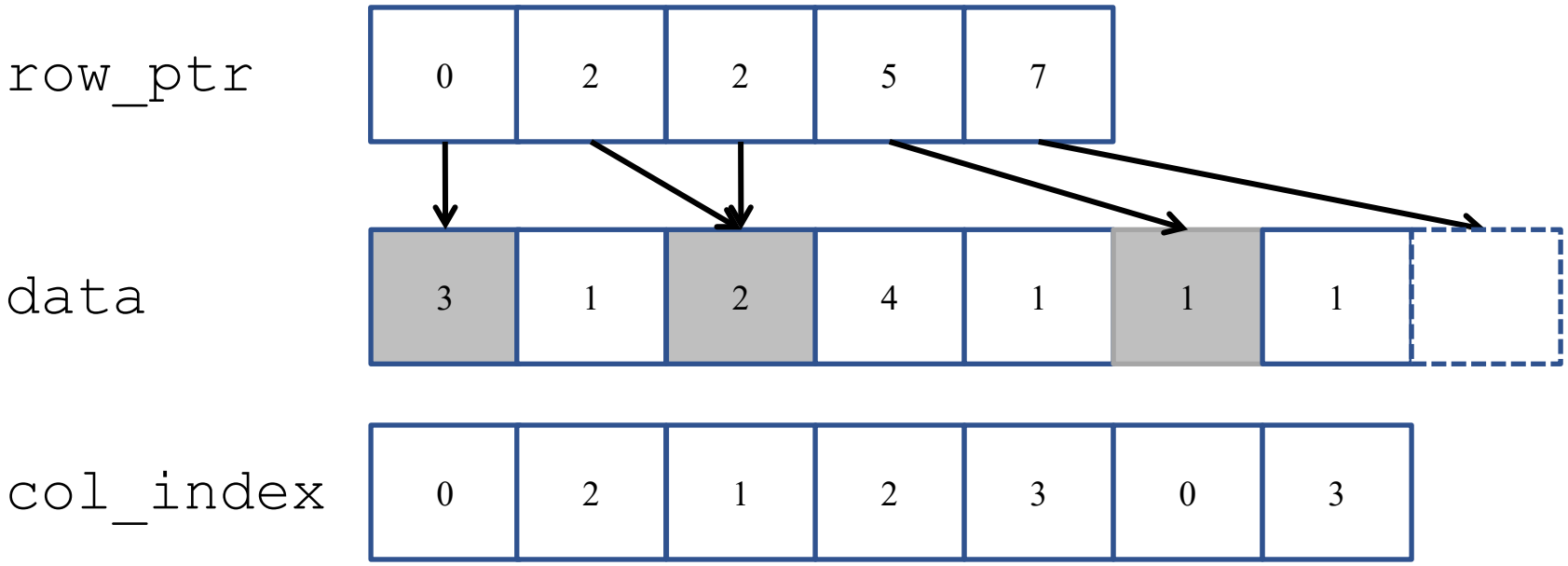
- Threads execute different number of iterations

Row 0	3	0	1	0
Row 1	0	0	0	0
Row 2	0	2	4	1
Row 3	1	0	0	1



CSR Kernel Memory Divergence (Uncoalesced Accesses)

- Adjacent threads access non-adjacent memory locations
  - Grey elements are accessed by all threads in iteration 0



Compressed Sparse Column (CSC)

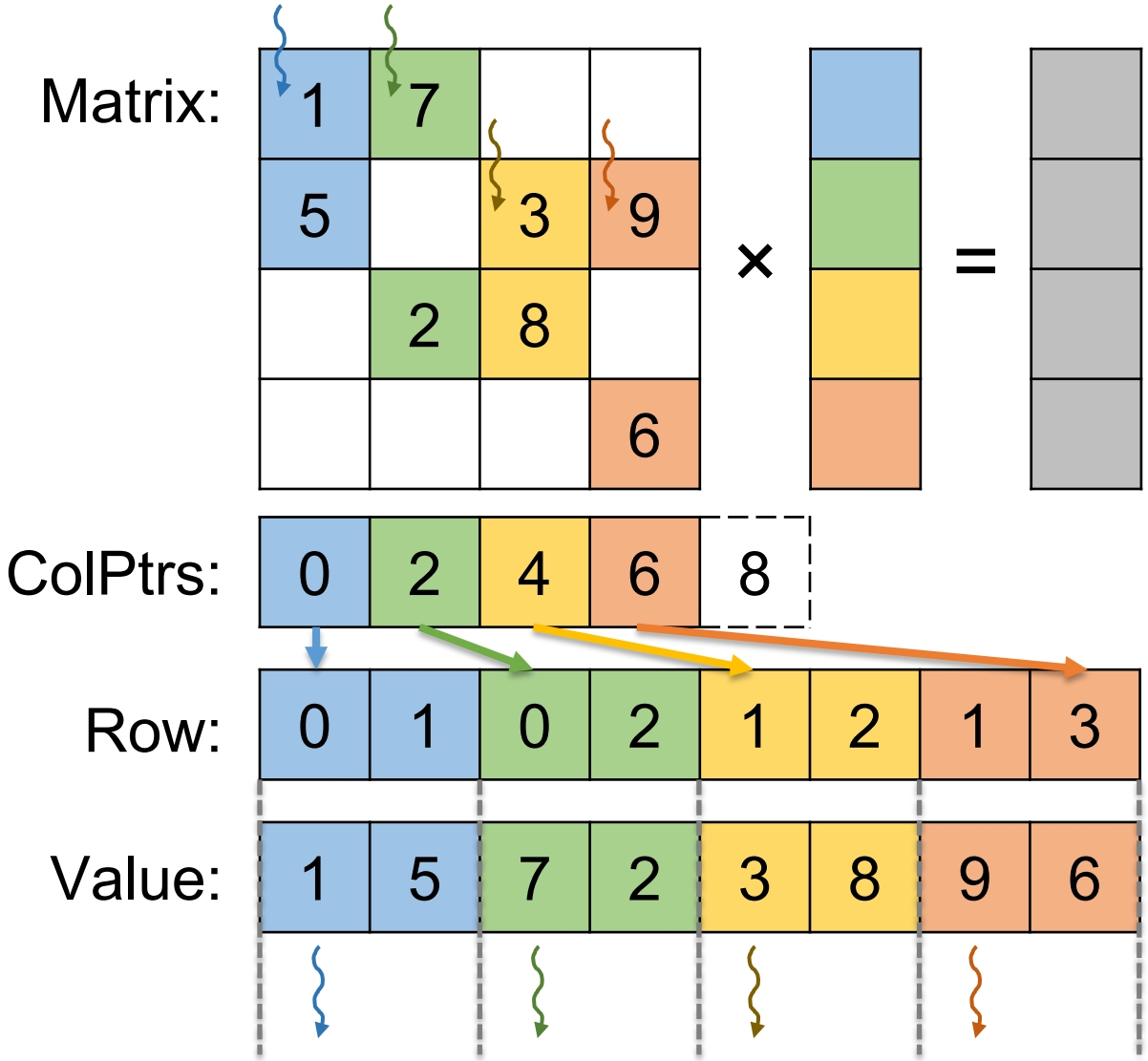
- Store nonzeros of the same column adjacently and an index to the first element of each column

Matrix:

1	7		
5		3	9
	2	8	
			6

ColPtrs:	0	2	4	6	8			
Row:	0	1	0	2	1	2	1	3
Value:	1	5	7	2	3	8	9	6

SpMV/CSC Parallelization



Parallelization approach:
Assign one thread to loop over each input column sequentially and update corresponding output elements

CSC Tradeoffs (versus CSR)

- Advantages:
  - Space efficiency: same as CSR
  - Accessibility: given a column, easy to find all nonzeros
  - SpMV/CSC achieves memory accesses to the input vector are coalesced

- Disadvantage:
  - Modifiability: hard to add new elements to the matrix
  - Accessibility: given nonzero, hard to find row; given a row, hard to find all nonzeros
  - SpMV/CSC memory accesses to the input matrix are not coalesced
  - SpMV/CSC memory accesses to the output vector require atomic operations
  - SpMV/CSC has control divergence

- CSC is typically not used for SpMV
  - Useful for computations where the vector is very sparse: if an input vector value is zero, we can avoid accessing and entire matrix column

Regularizing SpMV with ELL(PACK) Format

Matrix:

1	7		
5		3	9
	2	8	
			6

Group nonzeros by row (like CSR)…

Column:

0	1	
0	2	3
1	2	
3		

Value:

1	7	
5	3	9
2	8	
6		

ELLPACK Format (ELL)

Matrix:

1	7		
5		3	9
	2	8	
			6

…pad rows so they all have the same size…

Column:

0	1	*
0	2	3
1	2	*
3	*	*

Value:

1	7	*
5	3	9
2	8	*
6	*	*

ELLPACK Format (ELL)

Matrix:

1	7		
5		3	9
	2	8	
			6

…and store padded array of nonzeros in column major order

Column:

0	1	*
0	2	3
1	2	*
3	*	*

Value:

1	7	*
5	3	9
2	8	*
6	*	*

ELLPACK Format (ELL)

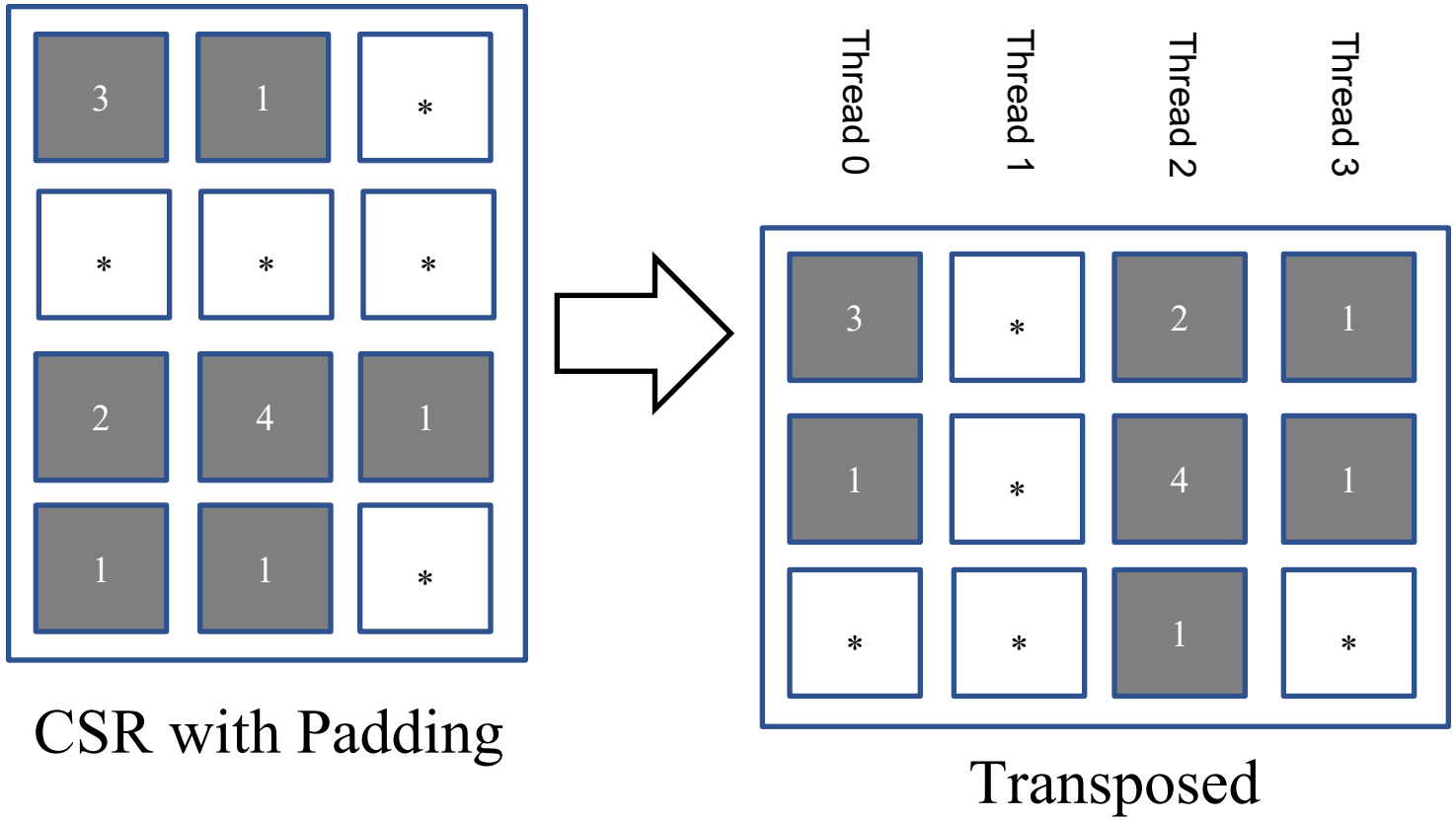
Matrix:

1	7		
5		3	9
	2	8	
			6

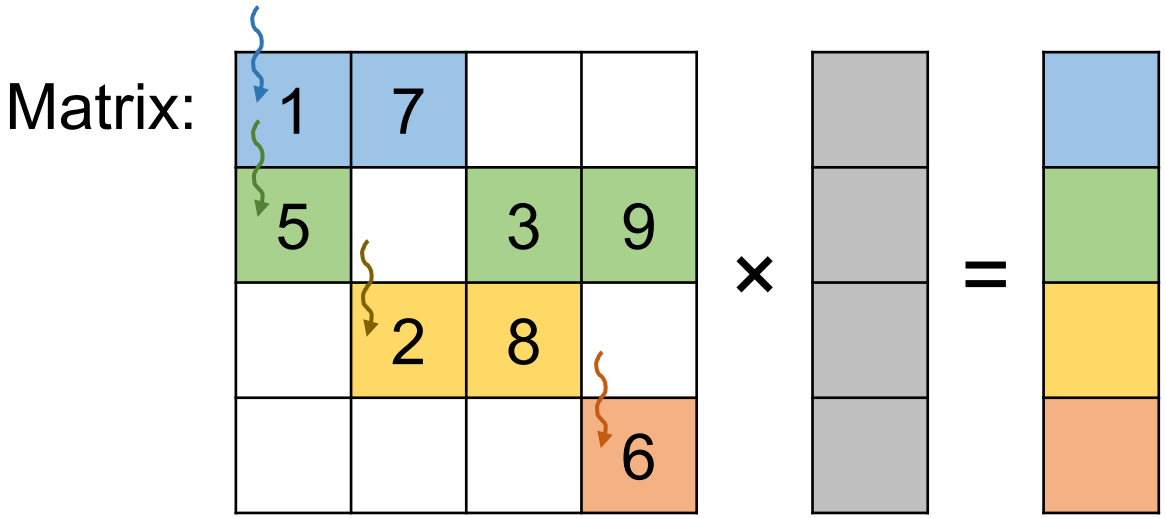
…and store padded array of nonzeros in column major order

Column:	0	0	1	3	1	2	2	*	*	3	*	*
Value:	1	5	2	6	7	3	8	*	*	9	*	*

Regularizing SpMV with ELL(PACK) Format



SpMV/ELL Parallelization



Parallelization approach:
Assign one thread to loop over each input row sequentially and update corresponding output element

Column:	0	0	1	3	1	2	2	*	*	3	*	*
Value:	1	5	2	6	7	3	8	*	*	9	*	*

Memory accesses are coalesced

SpMV/ELL Code

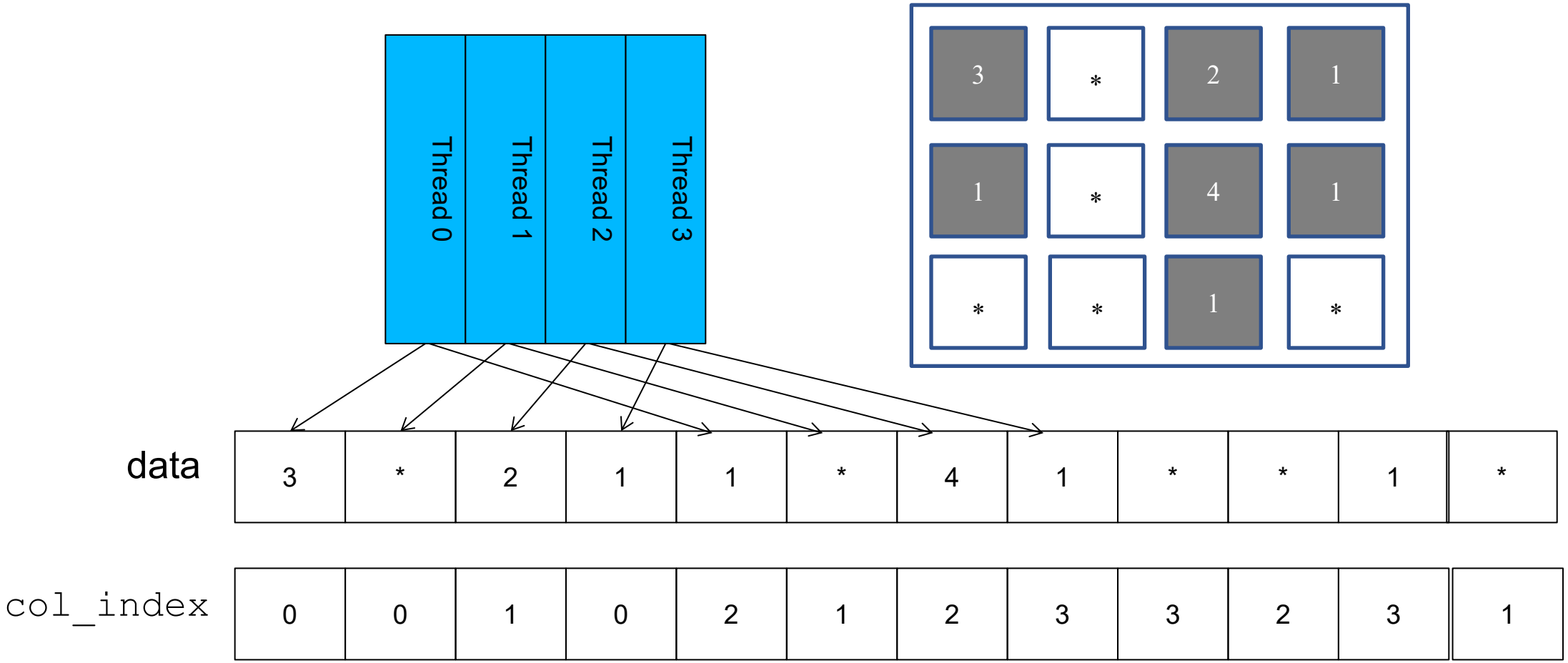
```
__global__ void SpMV_ELL(int num_rows, float *data,
     int *col_index, int num_elem, float *x, float *y)
 {
   int row = blockIdx.x * blockDim.x + threadIdx.x;
   if (row < num_rows) {
     float dot = 0;
     for (int i = 0; i < num_elem; i++)
        dot += data[row+i*num_rows]*x[col_index[row+i*num_rows]];
     y[row] = dot;
   }
 }
```

ELL Tradeoffs

- Advantages:
  - Modifiability: can add new elements as long as row not full
  - Accessibility: given a row, easy to find all nonzeros; given nonzero, easy to find row and column
  - SpMV/ELL memory accesses are coalesced

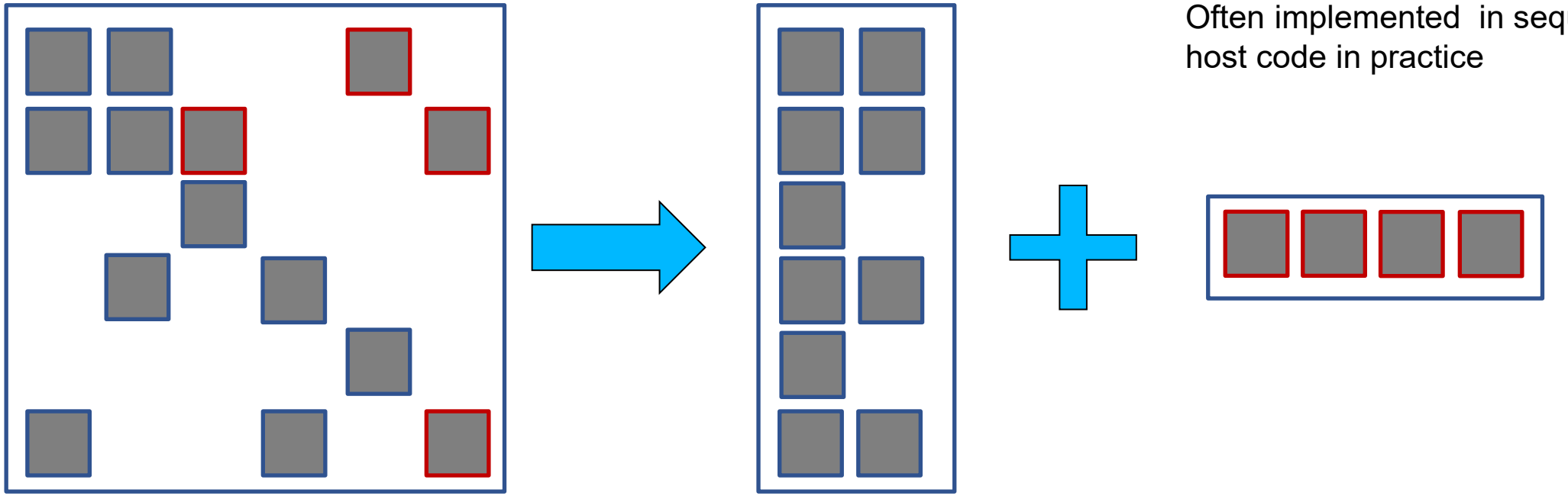
- Disadvantage:
  - Space efficiency: overhead due to padding
  - Accessibility: given a column, hard to find all nonzeros

Memory Coalescing with ELL



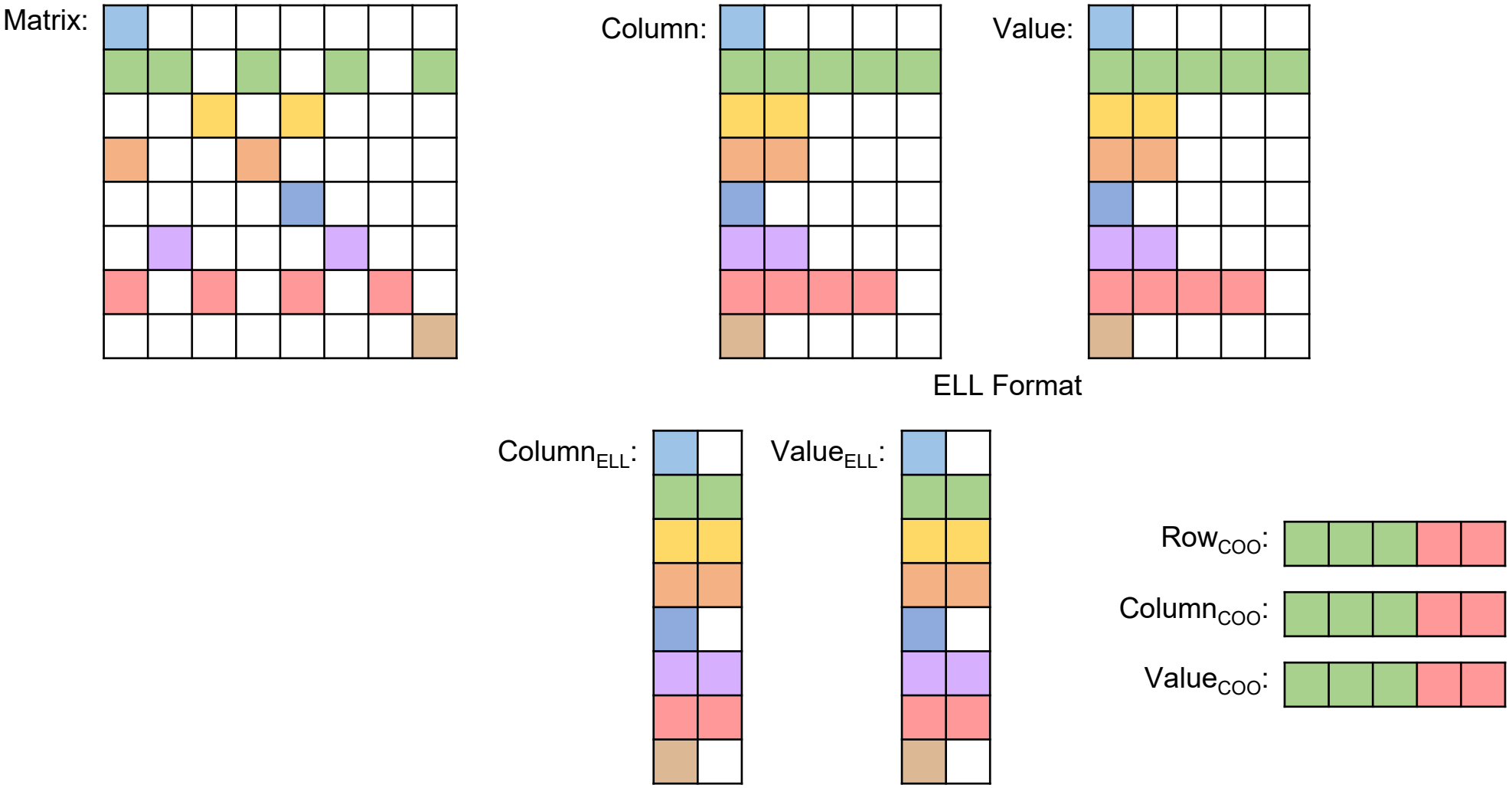
Hybrid Format (ELL + COO)

- ELL handles *typical* entries
- COO handles *exceptional* entries
  - Implemented with segmented reduction



Often implemented in sequential host code in practice

Hybrid ELL + COO Example



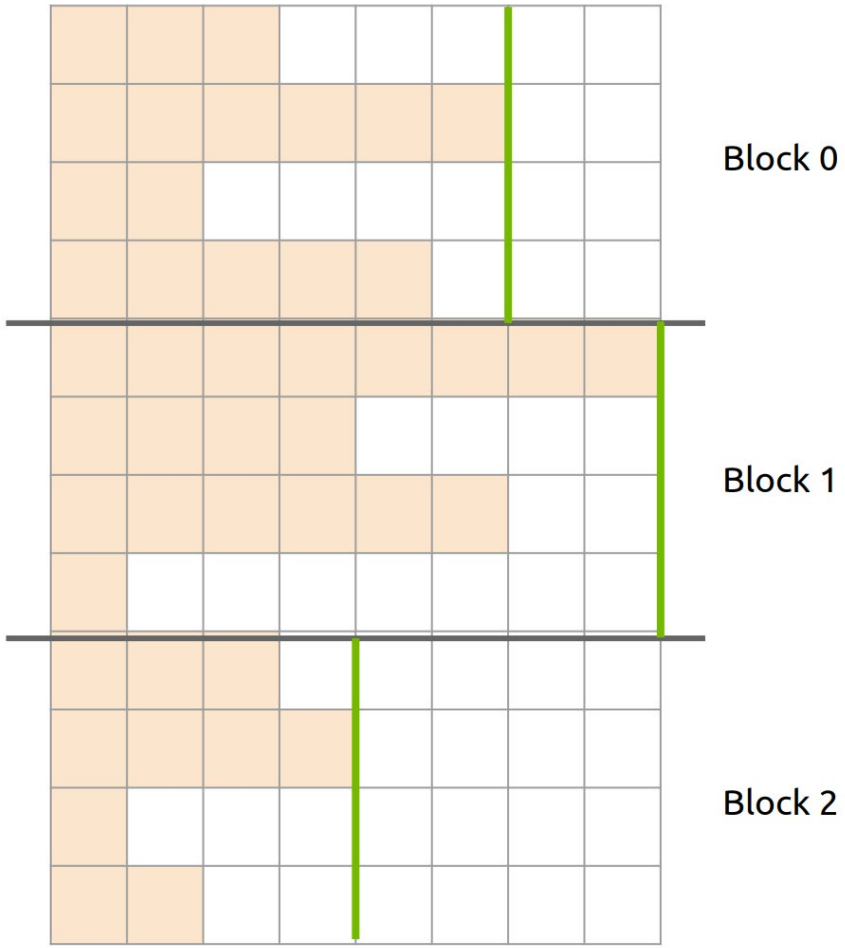
Hybrid ELL + COO
(use COO for very long rows)

ELL + COO Tradeoffs

- Similar to ELL, with the following added benefits from using COO:
  - Space efficiency: less padding
  - Modifiability: can add new elements to any row
  - Less control divergence

CSR Run-time

- Block performance is determined by longest row



Jagged Diagonal Sparse Kernel Design for Load Balancing

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

Group nonzeros by row (like CSR)…

Column:

0			
0	1	3	4
2	4		
0	3	5	
1	4		
0	2	5	

Value:

a			
b	c	d	e
f	g		
h	i	j	
k	l		
m	n	o	

Jagged Diagonal Storage (JDS)

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

…sort rows by size and remember the original row index…

Column:

0	1	3	4
0	3	5	
0	2	5	
2	4		
1	4		
0			

Value:

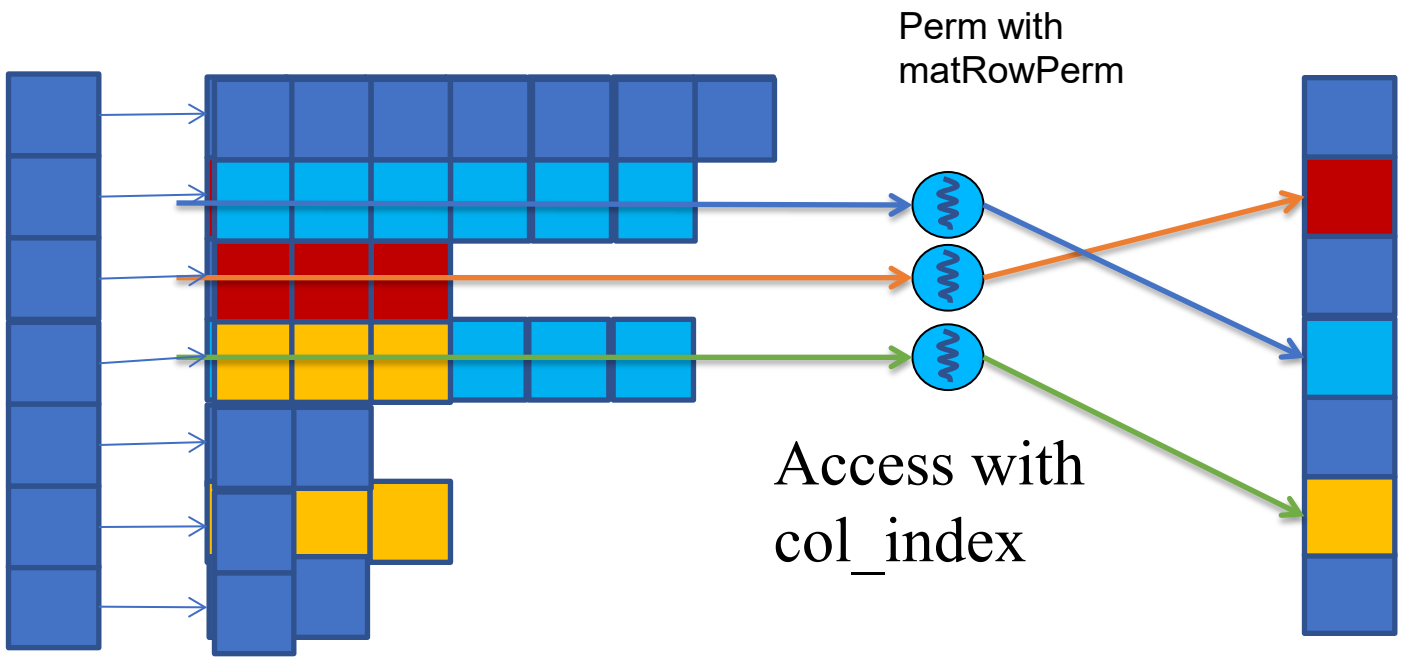
b	c	d	e
h	i	j	
m	n	o	
f	g		
k	l		
a			

Row:

1
3
5
2
4
0

Jagged Diagonal Storage (JDS)

- Keep track of the original row numbers so that the output vector can be generated correctly.



CSR to JDS Conversion

	Row 0	Row 2	Row 3
Nonzero values data[7]	{ 3, 1,	2, 4, 1,	1, 1 }
Column indices col_index[7]	{ 0, 2,	1, 2, 3,	0, 3 }

Row Pointers row_ptr[5] { 0, 2, 2, 5, 7 }

	Row 2	Row 0	Row 3
Nonzero values data[7]	{ 2, 4, 1,	3, 1,	1 1 }
Column indices col_index[7]	{ 1, 2, 3,	0, 2,	0, 3 }

JDS Row Pointers jds_row_ptr[5] {0, 3, 5, 7, 7 }

JDS Row Indices jds_row_perm[4] {2, 0, 3, 1 }

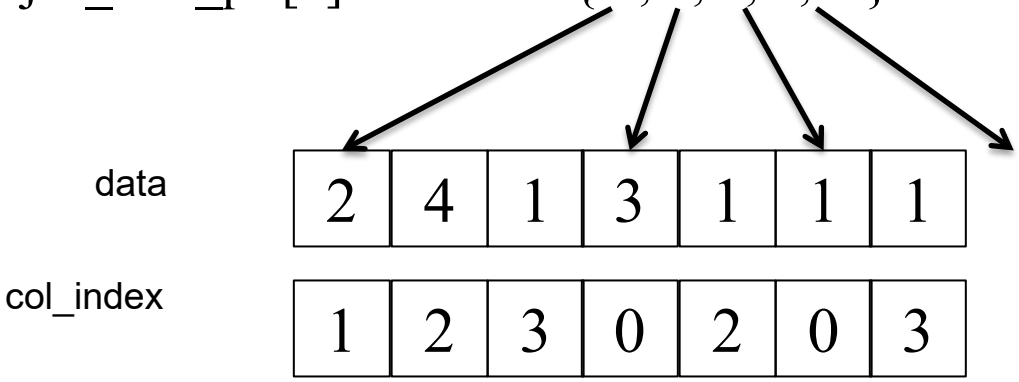
SpMV/JDS Code

```
__global__ void SpMV_JDS(int num_rows, float *data,  int *col_index,
                         int *jds_row_ptr, int *jds_row_perm, float *x, float *y)
{
  int row = blockIdx.x * blockDim.x + threadIdx.x;
  if (row < num_rows) {
    float dot = 0;
    int row_start = jds_row_ptr[row];
    int row_end =   jds_row_ptr[row+1];
    for (int elem = row_start; elem < row_end; elem++) {
      dot += data[elem] * x[col_index[elem]];
    }
    y[jds_row_perm[row]] = dot;
  }
}
```

JDS vs. CSR - Control Divergence

- Threads still execute different number of iterations in the JDS kernel for-loop
  - However, neighboring threads tend to execute similar number of iterations because of sorting.
  - Better thread utilization, less control divergence

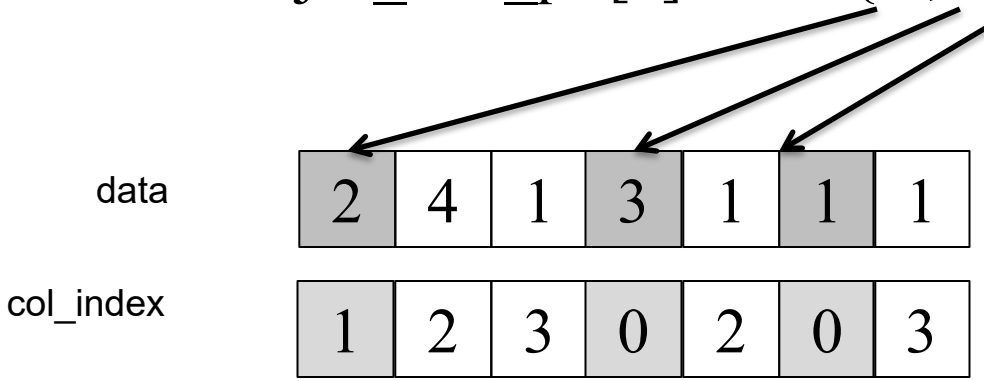
Nonzero values	data[7]	{ 2, 4, 1, 3, 1, 1, 1 }
Column indices	col_index[7]	{ 1, 2, 3, 0, 2, 0, 3 }
JDS row indices	jds_row_perm[4]	{ 2, 0, 3, 1 }
JDS Row Ptrs	jds_row_ptr[5]	{ 0, 3, 5, 7, 7 }



JDS vs. CSR Memory Divergence

- Adjacent threads still access non-adjacent memory locations

Nonzero values	data[7]	{ 2, 4, 1, 3, 1, 1, 1 }
Column indices	col_index[7]	{ 1, 2, 3, 0, 2, 0, 3 }
JDS row indices	jds_row_perm[4]	{ 2, 0, 3, 1 }
JDS Row Ptrs	jds_row_ptr[5]	{ 0, 3, 5, 7, 7 }



JDS with Transposition

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

…store nonzeros in column major…

Column:

0	1	3	4
0	3	5	
0	2	5	
2	4		
1	4		
0			

Value:

b	c	d	e
h		i	
m	n	o	
f	g		
k	l		
a			

Row:

1
3
5
2
4
0

JDS with Transposition

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

…store nonzeros in column major…

Row:

1
3
5
2
4
0

Column:

0	0	0	2	1	0	1	3	2	4	4	3	5	5	4

Value:

b	h	m	f	k	a	c	i	n	g	l	d	j	o	e

JDS with Transposition

Matrix:

a					
b	c		d	e	
		f		g	
h			i		j
	k			l	
m		n			o

…and remember where the nonzeros of each iteration start

IterPtr:

0	6	11	14	15

Column:

0	0	0	2	1	0	1	3	2	4	4	3	5	5	4

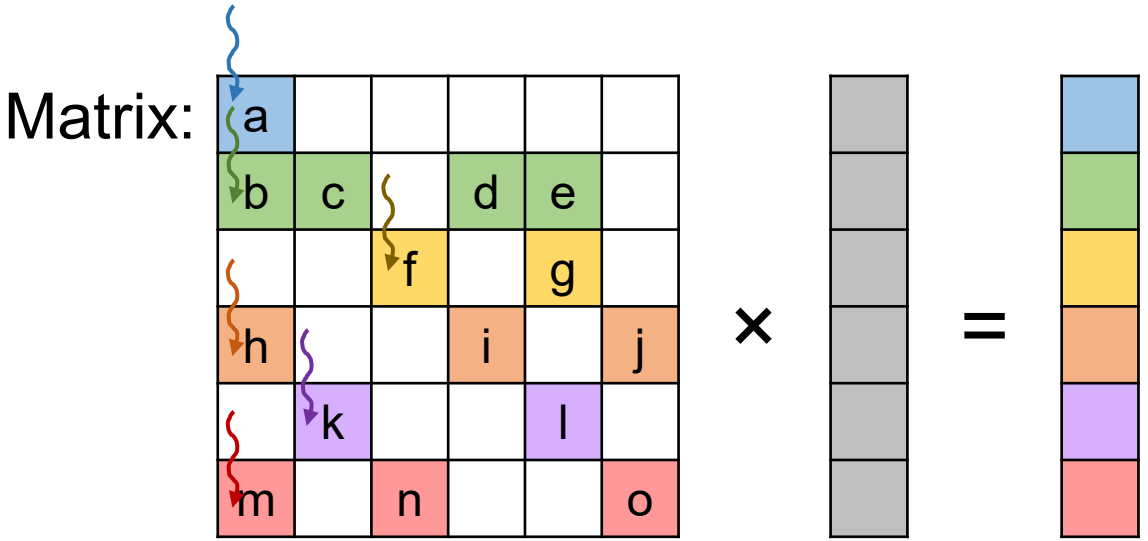
Value:

b	h	m	f	k	a	c	i	n	g	l	d	j	o	e

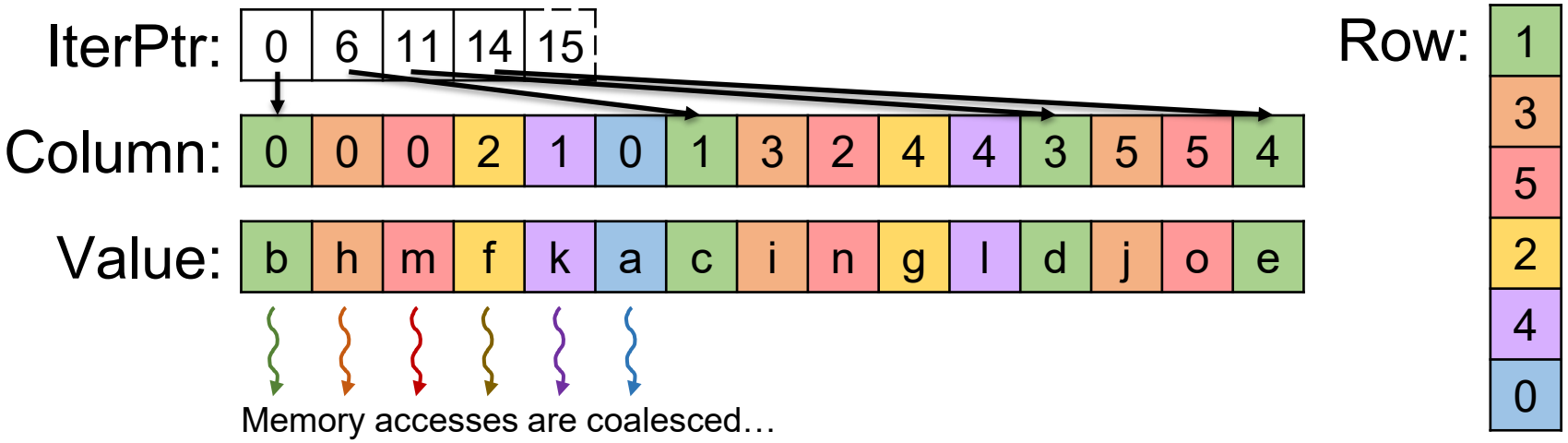
Row:

1
3
5
2
4
0

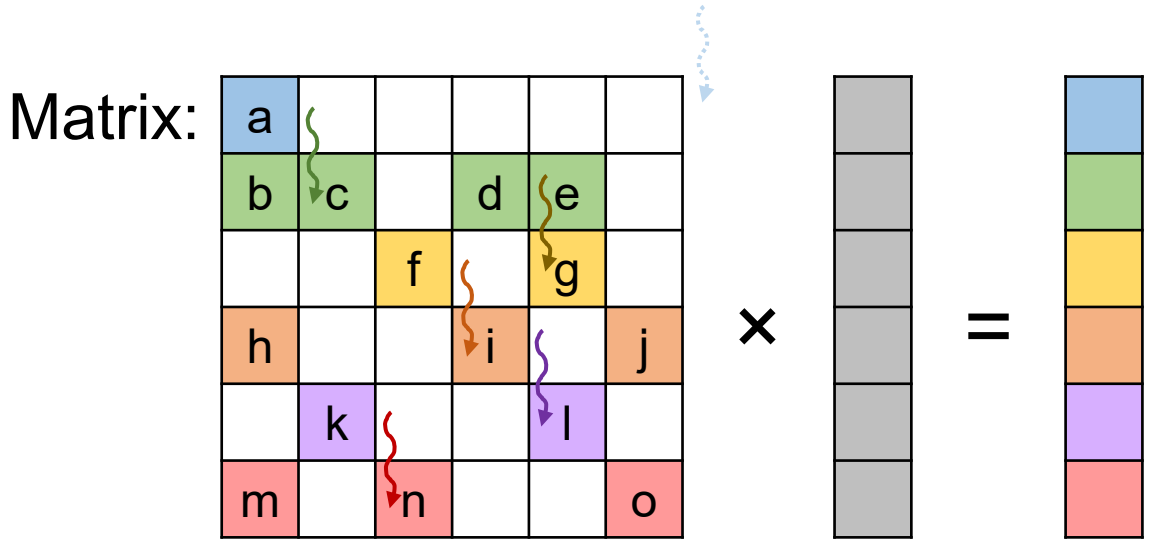
SpMV/JDS Parallelization



Parallelization approach:
Assign one thread to loop over each input row sequentially and update corresponding output element



SpMV/JDS Parallelization



Parallelization approach:
Assign one thread to loop over each input row sequentially and update corresponding output element

IterPtr:	0	6	11	14	15										
Column:	0	0	0	2	1	0	1	3	2	4	4	3	5	5	4
Value:	b	h	m	f	k	a	c	i	n	g	l	d	j	o	e

Row:
1
3
5
2
4
0

Memory accesses are coalesced…
…and threads drop out from the end, minimizing control divergence

SpMV/JDS_T Code

```
__global__ void SpMV_JDS_T(int num_rows, float *data, int *col_index,
            int *jds_t_col_ptr, int *jds_row_perm, float *x, float *y)
{
    int row = blockIdx.x * blockDim.x + threadIdx.x;
    if (row < num_rows) {
        float dot = 0;
        unsigned int sec = 0;
        while (jds_t_col_ptr[sec+1] - jds_t_col_ptr[sec] > row) {
            dot += data[jds_t_col_ptr[sec]+row] *
                   x[col_index[jds_t_col_ptr[sec]+row]];
            sec++;
        }
        y[jds_row_perm[row]] = dot;
    }
}
```

Lab 8 Variable Names

JDS_T Length of Cols matRows[4]	{3,	2,	2,	0 }

	Sec 0	Sec 1	Sec 2
Nonzero values matData[7]	{ 2, 3, 1,	4, 1, 1	1 }
Column indices matCols[7]	{ 1, 0, 0,	2, 2, 3	3 }

JDS_T Column Pointers matColStart[4]	{0,	3,	6,	7 }
JDS Row Indices matRowPerm[4]	{2,	0,	3,	1 }

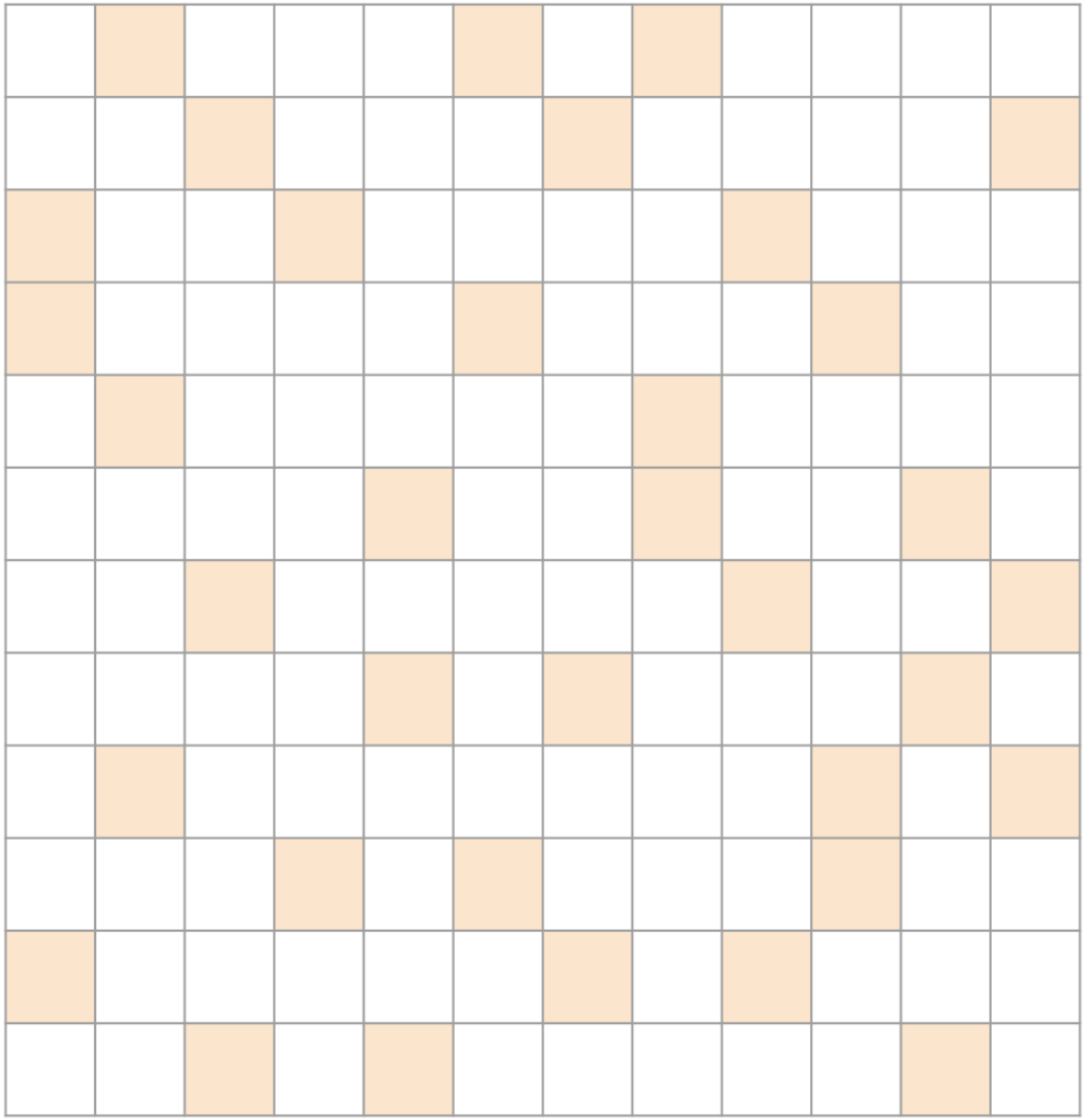
JDS Tradeoffs

- Advantages:
  - Space efficiency: no padding
  - Accessibility: given a row, easy to find all nonzeros
  - SpMV/JDS memory accesses are coalesced
  - SpMV/JDS minimizes control divergence

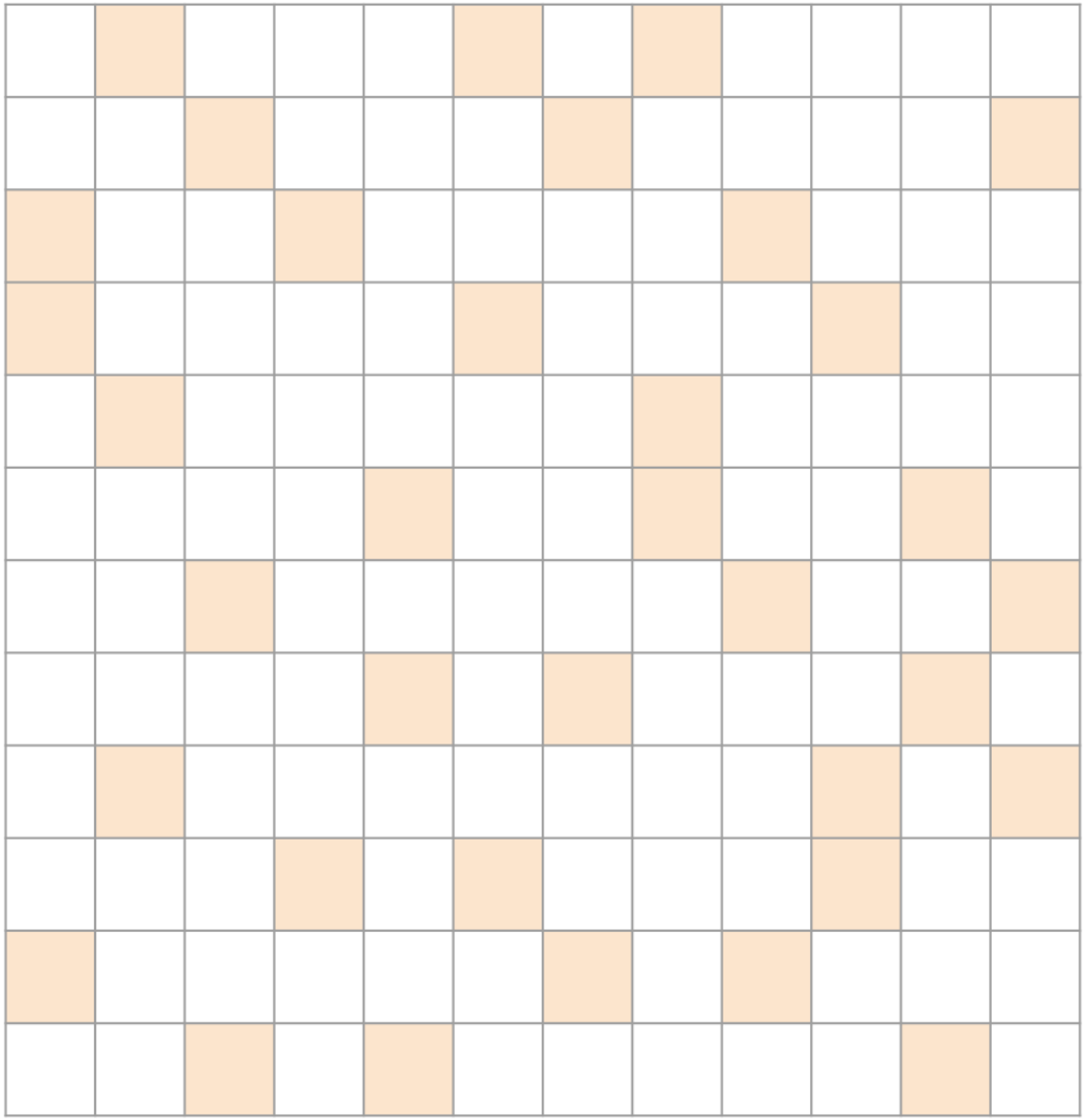
- Disadvantage:
  - Modifiability: hard to add new elements to the matrix
  - Accessibility: given nonzero, hard to find row; given a column, hard to find all nonzeros

What format should we use?

- For the next few examples of sparse matrixes, what format fits best?



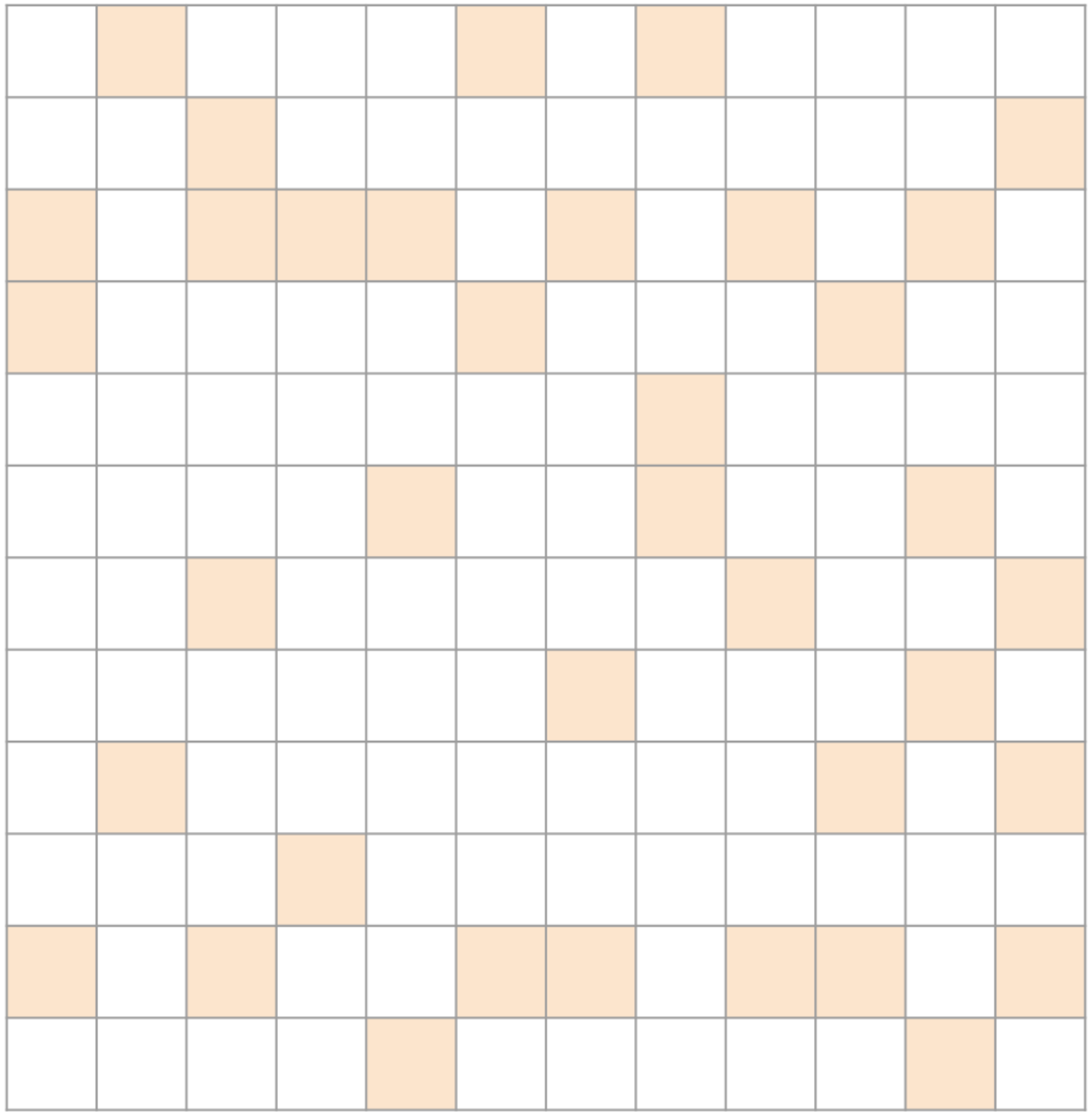
Roughly Random…



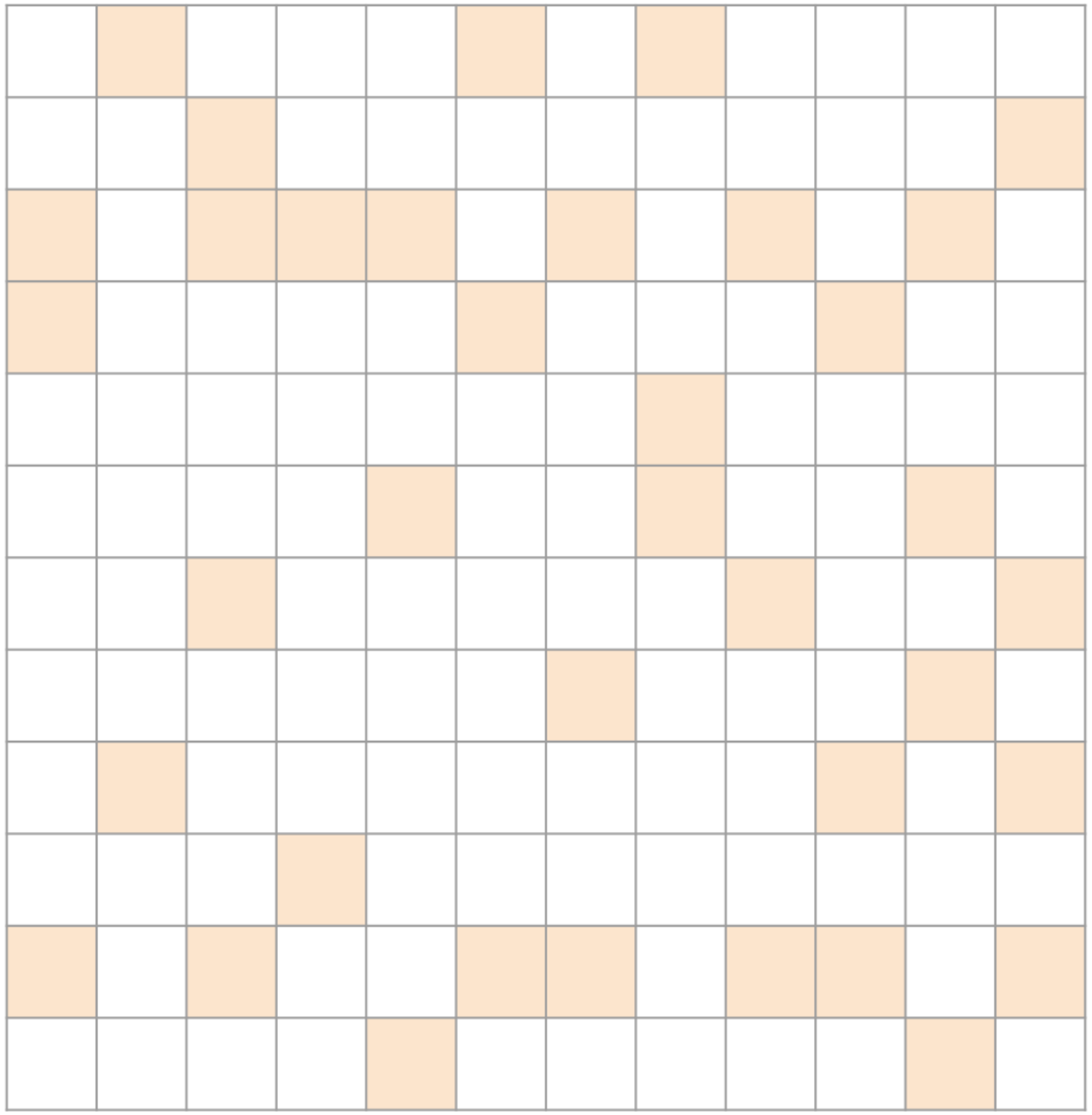
Roughly Random…

Probably best with ELL.

- Padding will be uniformly distributed
- Sparse representation will be uniform



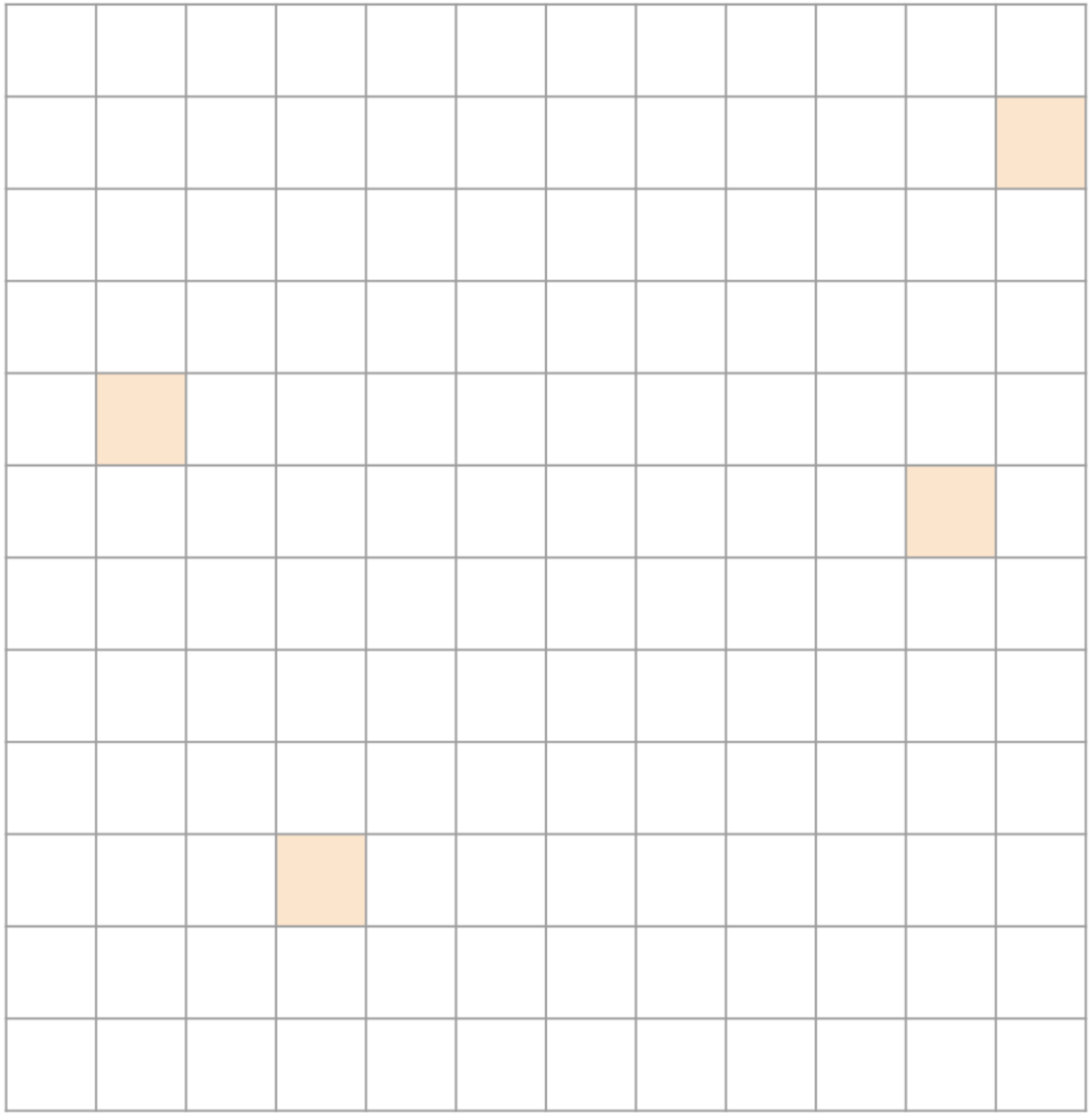
High variance in rows…



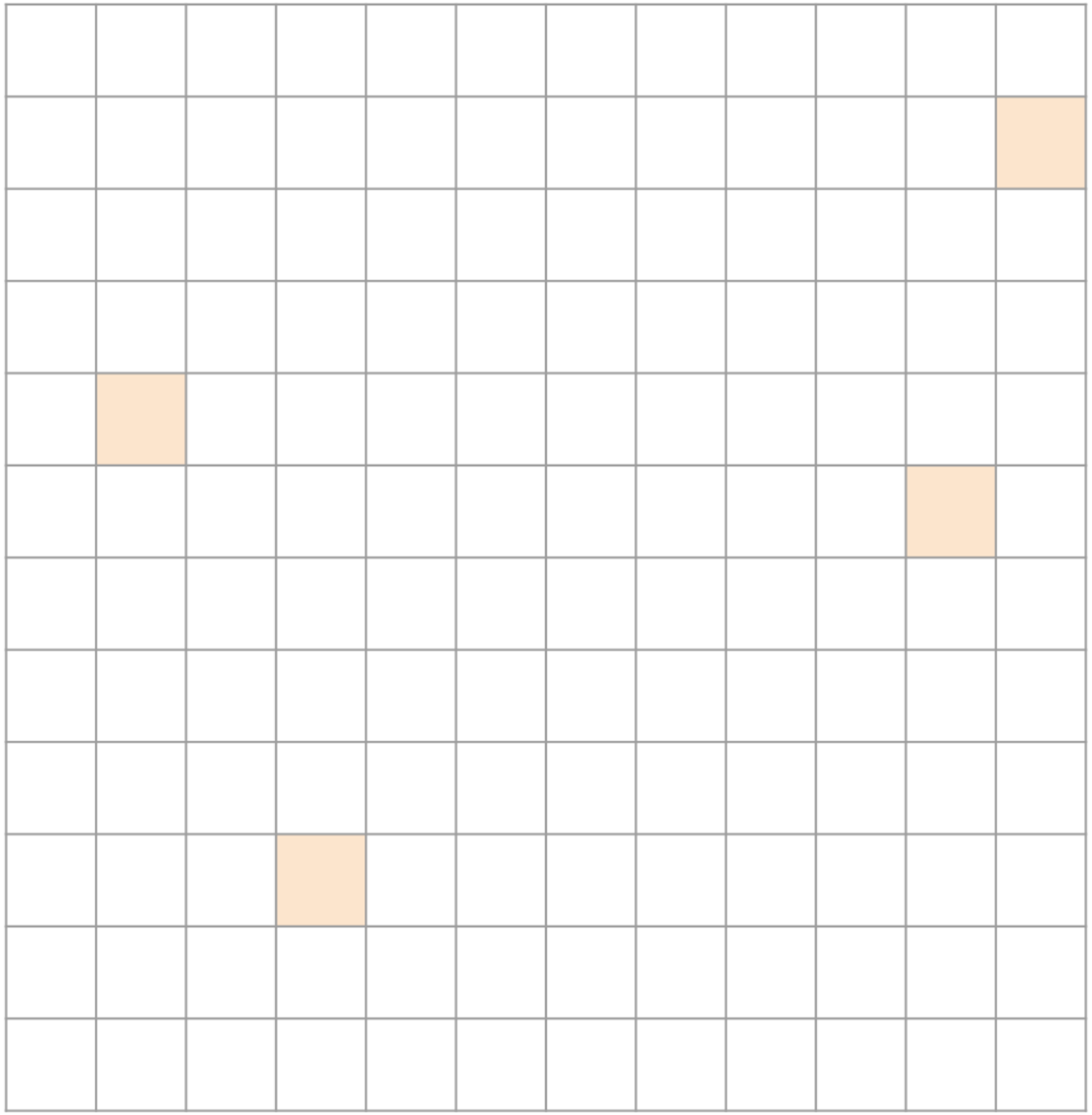
High variance in rows

Probably best with ELL/COO

- Benefit of ELL for most cases
- Outliers are captured with COO



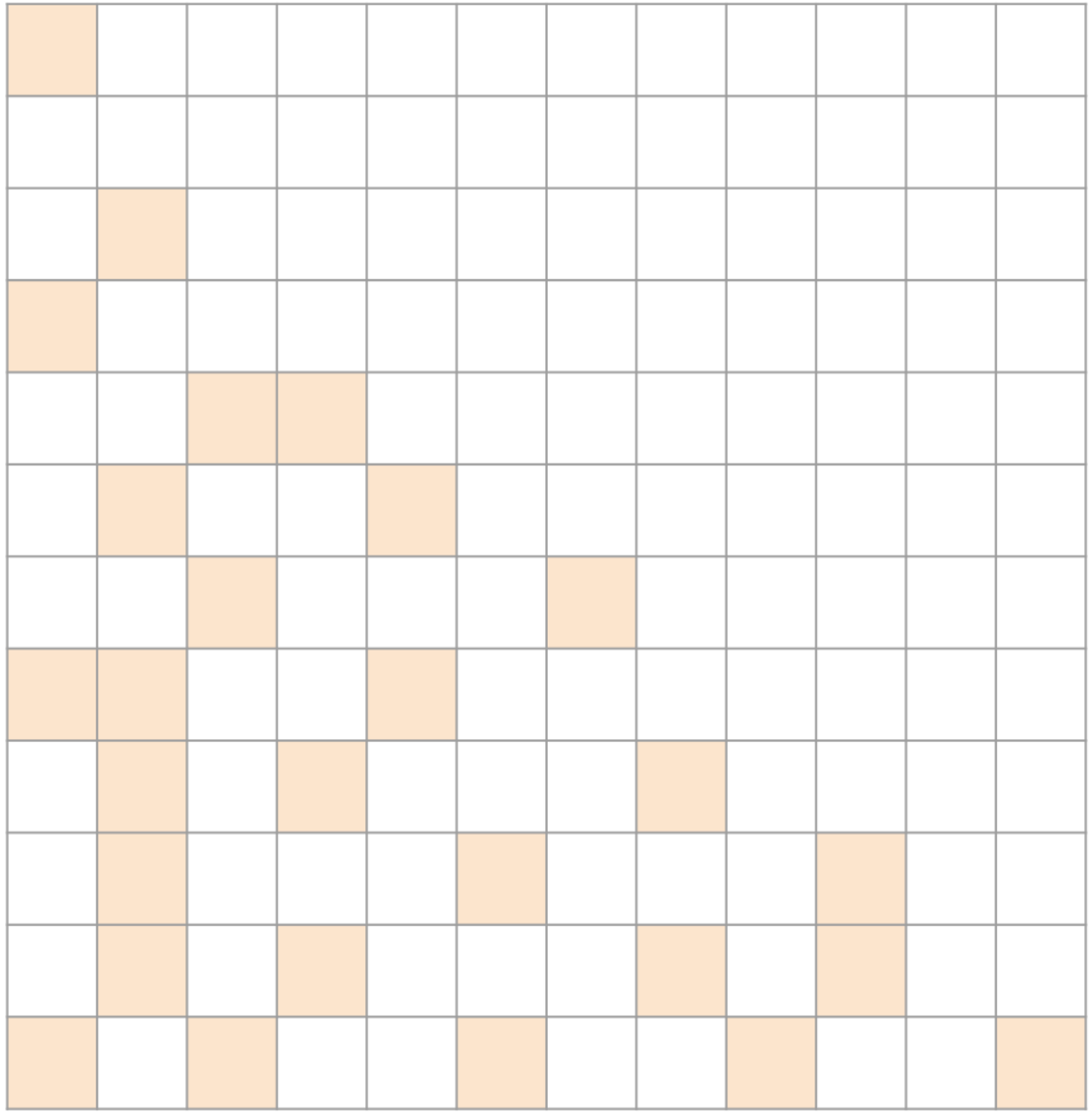
Very sparse…



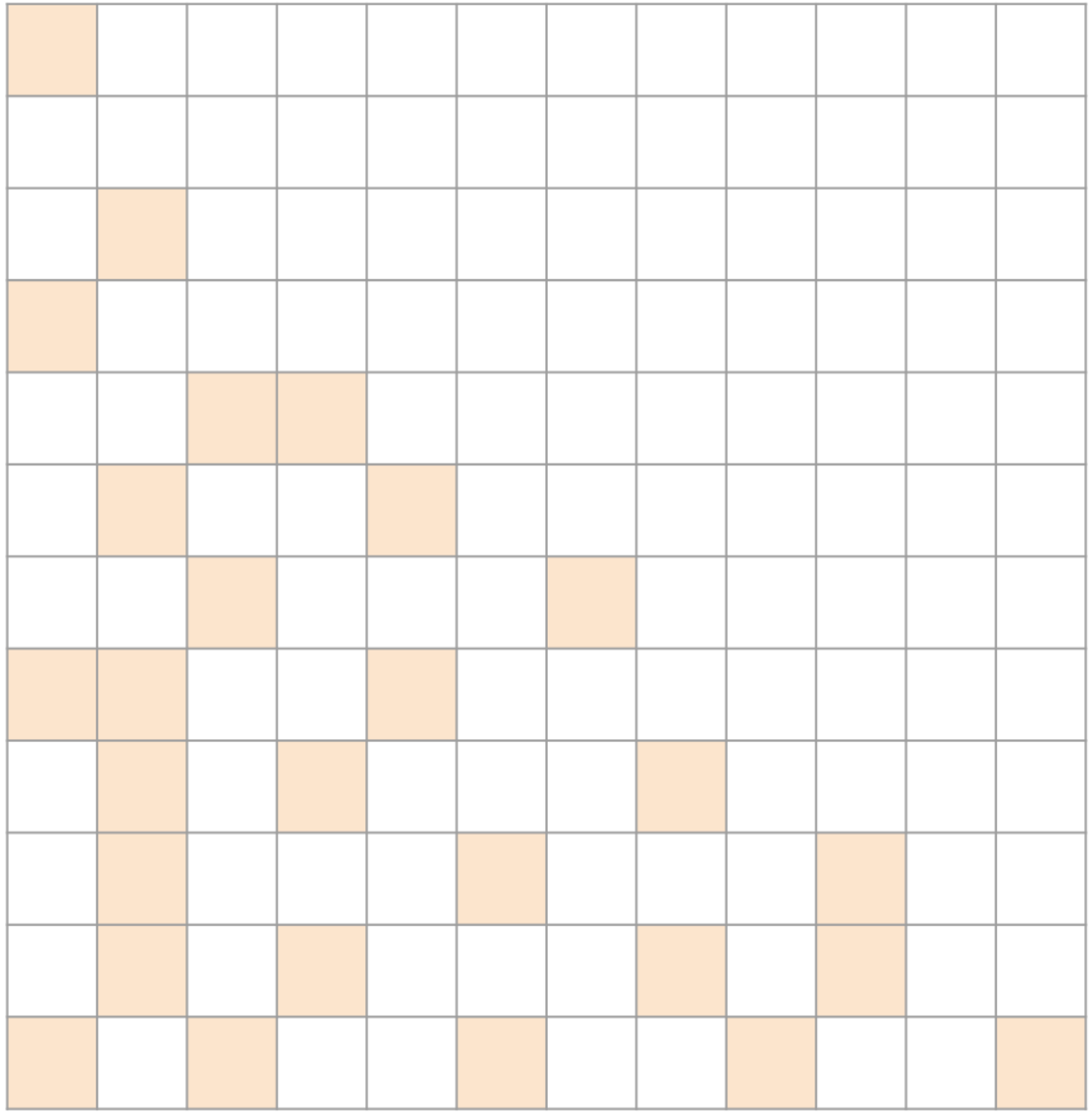
Very sparse

Probably best with COO

- Not a lot of data, compute is sparse



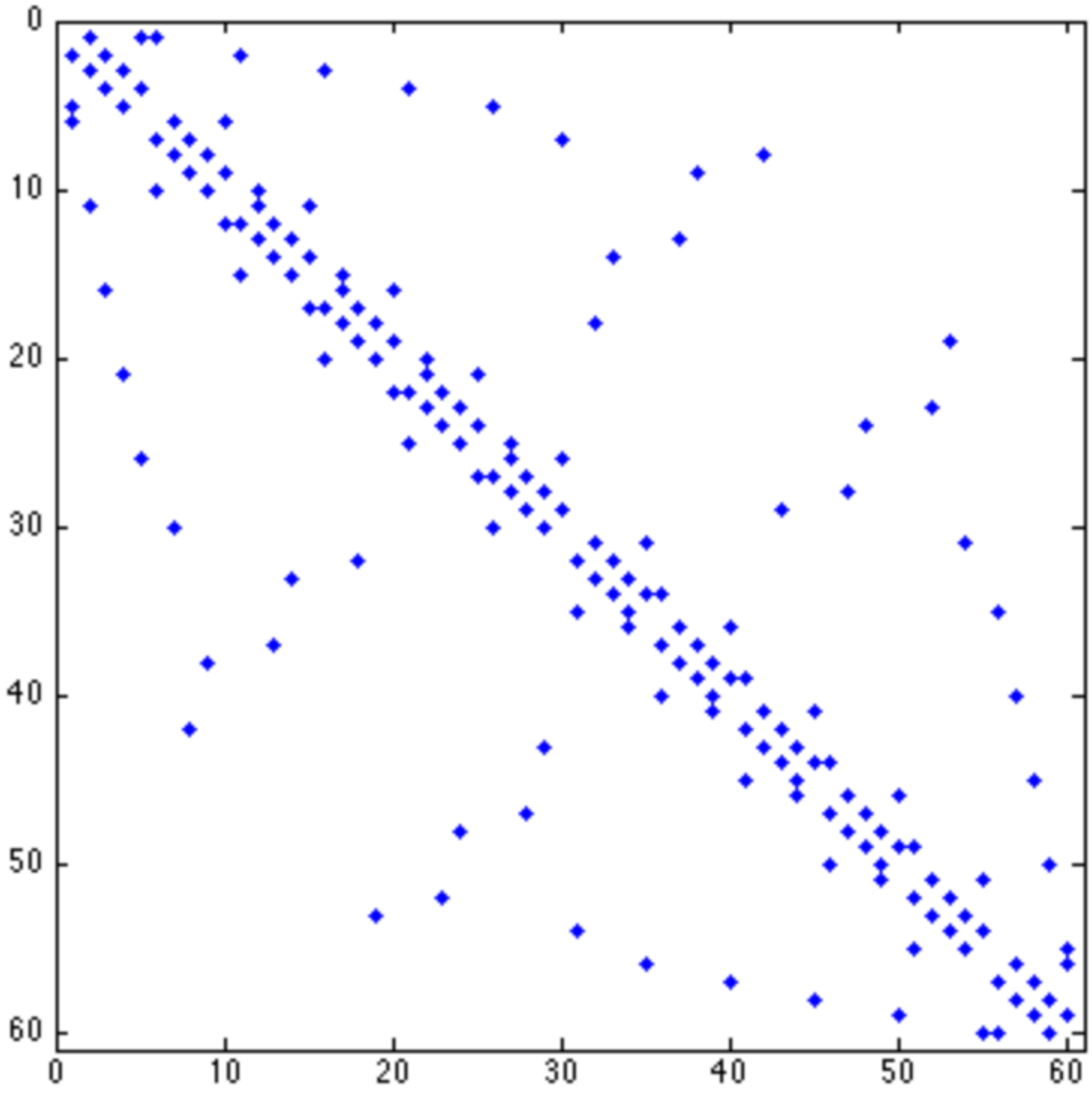
Roughly triangular…



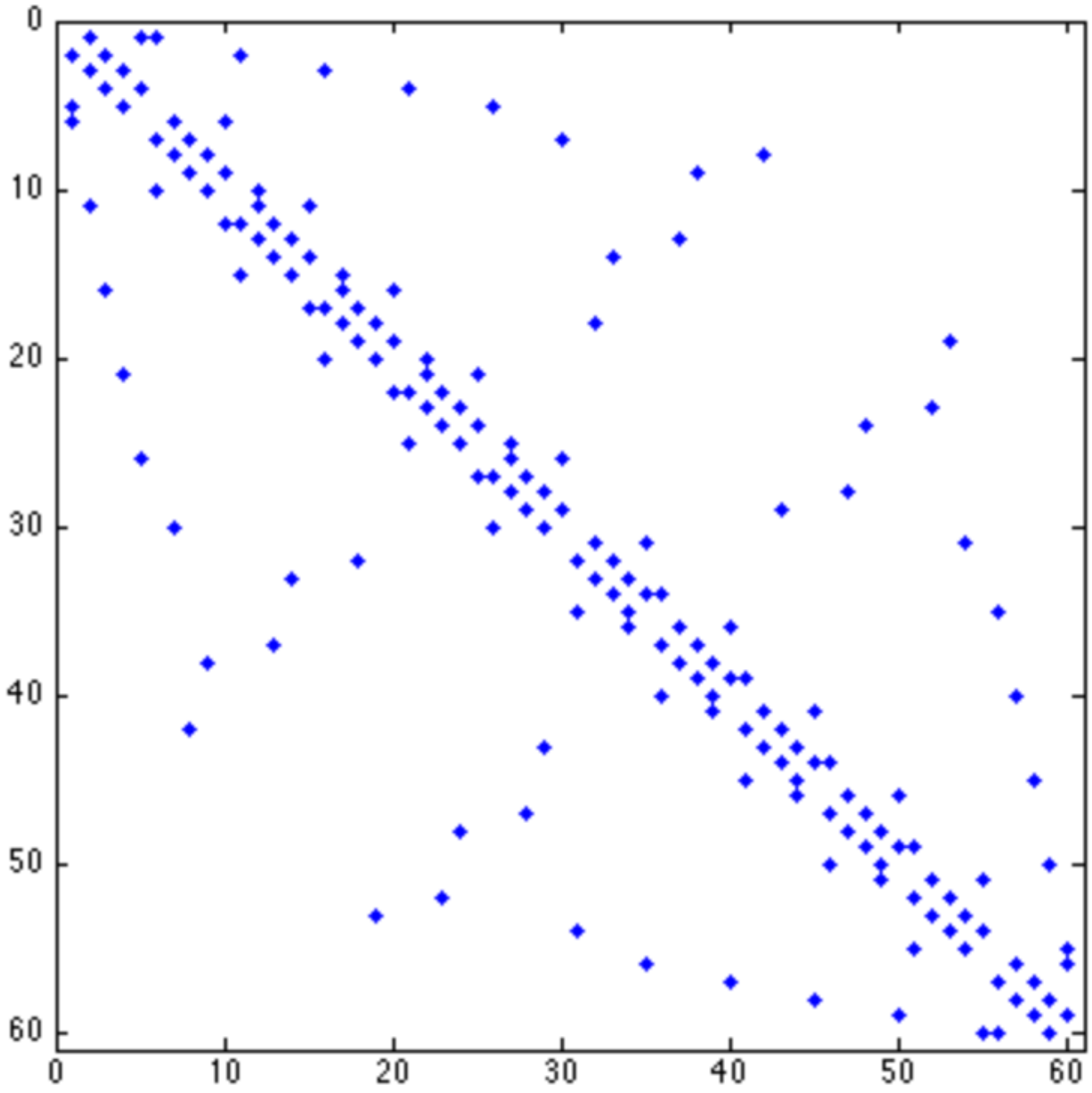
Roughly triangular…

Probably best with JDS

- Takes advantage of sparsity structure



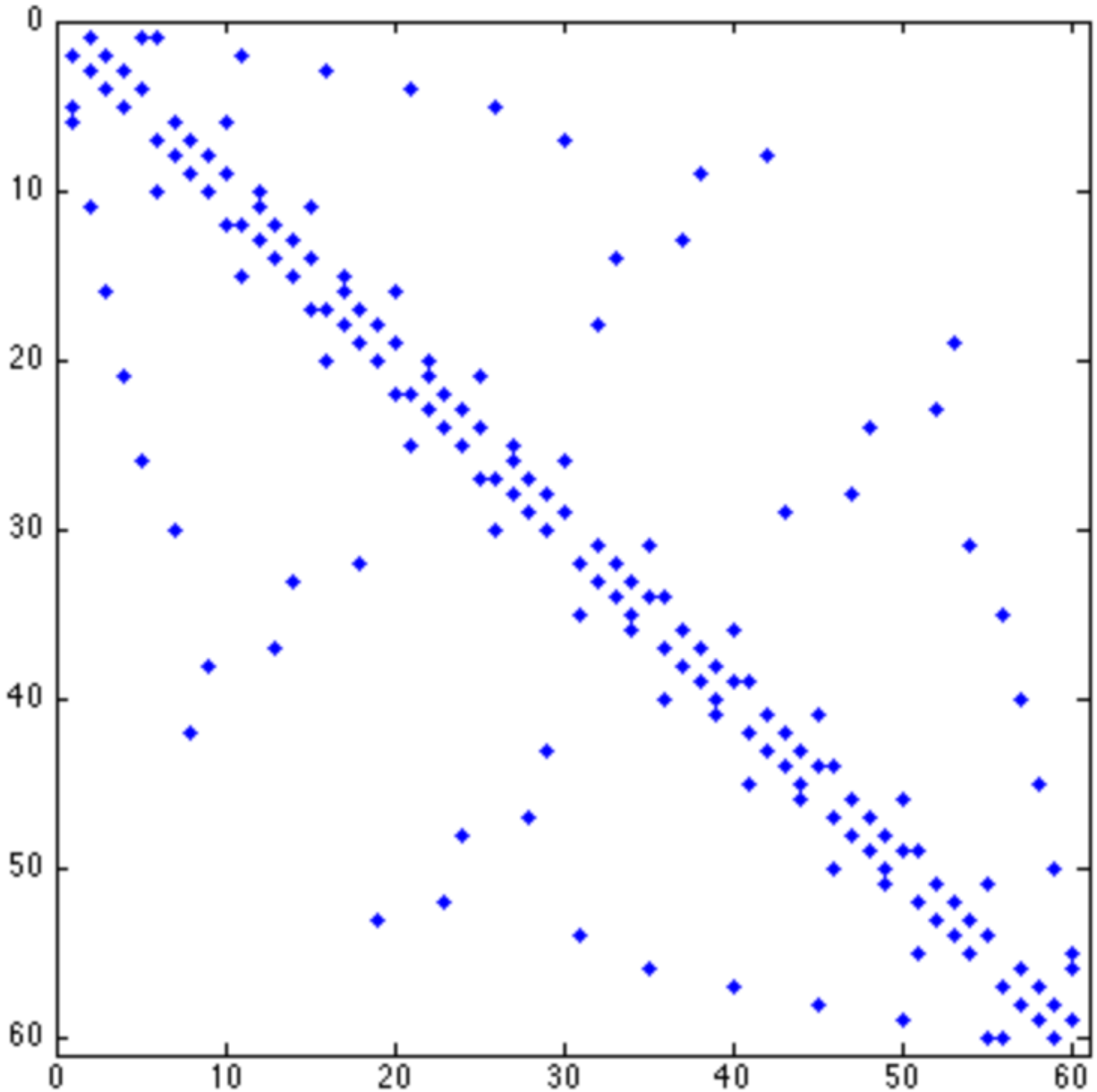
Banded Matrix…

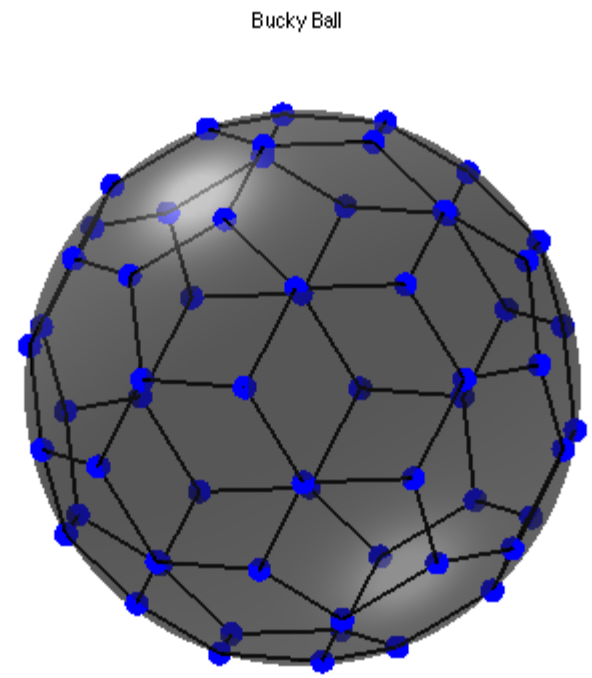


Banded Matrix…

Probably best with ELL

- Small amount of variance in rows


0
10
20
30
40
50
60
0
10
20
30
40
50
60


Bucky Ball

Other formats

- Diagonal (DIA): for strictly banded/diagonal matrices
- Packet (PKT): create diagonal submatrices by reordering rows/cols
- Dictionary of Keys (DOK): map of (row/col) to data
- Compressed Sparse Column (CSC): when to use over CSR?
- Blocked CSR: useful for block sparse matrices
- Hybrids of these…

Sparse Matrices as Foundation for Advanced Algorithms

- Graphs are often represented as sparse adjacency matrices
  - Used extensively in social network analytics, natural language processing, etc.
  - Sparse Matrix-Matrix multiplication (SpMM) is a fundamental operator in GNNs, which performs a multiplication between a sparse matrix and a dense matrix.
- Binning techniques often use sparse matrices for data compaction
  - Used extensively in ray tracing, particle-based fluid dynamics methods, and games
- These will be covered in ECE508/CS508

Any More Questions?

Things to Read:
Textbook Chapter 14

Things to Do:
Project CP2
Lab 8

Problem Solving

- Q: Given matrix A,
  which of the following are correct?

$$A = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 1 \\ 1 & 2 & 3 & 4 \\ 1 & 0 & 0 & 1 \end{vmatrix}$$

<u>CSR representation</u>

Data = [1,2,1,1,2,3,4,1,1]

Col_idx = [0,2,3,0,1,2,3,0,3]

Row_ptr = [0,1,3,7,9]

<u>COO representation</u>

Data = [1,2,1,1,2,3,4,1,1]

Col_idx = [0,2,3,0,1,2,3,0,3]

Row_idx = [0,1,1,3,3,3,3,7,7]

- A: only CSR

Problem Solving

$$\begin{bmatrix} 1 & 0 & 4 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 \\ 0 & 7 & 0 & 9 & 3 \\ 0 & 0 & 0 & 0 & 0 \\ 6 & 5 & 0 & 0 & 8 \end{bmatrix}$$

- Q: Consider the following sparse Matrix:
- For each of the following **data** layouts in memory, select the option that best matches all the sparse matrix formats that can store the data in memory as depicted.

Layout 1:

1	4	2	7	9	3	6	5	8

Layout 2:

1	2	7	6	4	0	9	5	0	0	3	8

Layout 3:

7	9	3	6	5	8	1	4	2

Layout 4:

9	7	1	2	4	3	5	8	6

Layout 5:

7	6	1	2	9	5	4	3	8

- A:
  - 1) CSR, COO
  - 2) ??
  - 3) JDS, COO
  - 4) COO
  - 5) JDS-Transposed, COO